НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»

Факультет інформатики та обчислювальної техніки

Кафедра інформаційних систем та технологій

До захисту допущено:

Завідувач кафедри

_____ Олександр РОЛІК

«___»_____2025 р.

Дипломний проєкт

на здобуття ступеня бакалавра
за освітньо-професійною програмою «Інформаційне забезпечення робототехнічних систем»
спеціальності 126 «Інформаційні системи та технології»
на тему: «Роботизована система сортування сміття»

Виконала:
студентка IV курсу, групи ІК-13
Килівник Владислава Леонідівна _____

Керівник:
доцент кафедри ІСТ, к.т.н. доц.
Ткач Михайло Мартинович _____

Рецензент:
доцент, канд. техн. наук, доцент каф. ІПІ
Ліхоузова Тетяна Анатоліївна _____

Засвідчую, що у цьому дипломному проєкті немає запозичень з праць інших авторів без відповідних посилань.
Студентка _____

Київ – 2025 року

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Рівень вищої освіти – перший (бакалаврський)

Спеціальність – 126 «Інформаційні системи та технології»

Освітньо-професійна програма «Інформаційне забезпечення робототехнічних систем»

ЗАТВЕРДЖУЮ
Завідувач кафедри
_____ Олександр РОЛІК
«___»_____2025 р.

ЗАВДАННЯ
на дипломний проєкт студентці
Килівник Владиславі Леонідівні

1. Тема проєкту «Роботизована система сортування сміття», керівник проєкту Ткач Михайло Мартинович, доцент, к.т.н., затверджені наказом по університету від «23» травня 2025 р. № 1705-с

2. Термін подання студентом проєкту: 09.06.2025 р.

3. Вихідні дані до проєкту: система повинна забезпечувати точність сортування сміття більше 60%. Система повинна за 1-2 секунди визначати і підписувати, який це об’єкт. Набір зображень різних об’єктів сміття.

4. Зміст пояснювальної записки: проаналізувати предметну область та існуючі рішення задачі сортування сміття, побудувати і навчити модель на обраному датасеті, програмно реалізувати систему та інтерфейс для роботизованої системи сортування сміття і зробити висновки щодо її роботи.

5. Перелік графічного матеріалу: схема алгоритм навчання мережі (А3), схема алгоритму роботи системи (А3), діаграма варіантів використання (А3), діаграма послідовностей (А3).

6. Дата видачі завдання: «1» грудня 2024 року.

Календарний план

№ з/п	Назва етапів виконання дипломного проєкту	Термін виконання етапів проєкту	Примітка
1	Дослідження актуальності проблеми, огляд літератури та формування мети	17.04.2025	Виконано
2	Огляд існуючих рішень для вирішення задачі сортування сміття	23.04.2025	Виконано
3	Вибір технологій для вирішення задачі сортування сміття	29.04.2025	Виконано
4	Вибір датасету та вибір і навчання моделі	10.05.2025	Виконано
5	Програмна реалізація системи	30.05.2025	Виконано
6	Розробка документації	08.06.2025	Виконано

Студентка Владислава КИЛІВНИК

Керівник Михайло ТКАЧ

АНОТАЦІЯ

Роботизована система сортування сміття.

Проєкт містить 63 с. тексту, 21 рисунків, 2 таблиці, посилання на 22 літературні джерела, 1 додаток та 4 конструкторських документів.

Ключові слова: ГЛИБОКЕ НАВЧАННЯ, ЗГОРТКОВА НЕЙРОННА МЕРЕЖА, КОМП'ЮТЕРНИЙ ЗІР, МОБІЛЬНИЙ РОБОТ, РОБОТОТЕХНІКА, СИСТЕМА, СОРТУВАННЯ СМІТТЯ, ШТУЧНИЙ ІНТЕЛЕКТ.

Об'єктом розробки є роботизована система сортування сміття.

Мета розробки — підвищити якість розпізнавання сміття за рахунок розробки моделі класифікатора та системи, що його реалізує на базі згорткової нейронної мережі.

У дипломному проєкті розроблено систему, яка класифікує такі об'єкти як: батарейки, біологічний мусор, картон, одяг, скло, метал, папір, пластик, взуття у режимі реального часу на відео з веб-камери та з зображень. Для цього вирішено застосовати бібліотеку OpenCV, яка обробляє зображення у режимі реального часу на відео з веб-камери ноутбуку та розроблено програмний додаток з допомогою мови програмування Python і інтерфейс за допомогою бібліотеки Tkinter. І побудовано згорткову нейронну мережу для сортування сміття. Існує також потенціал для інтеграції системи в мобільного робота.

Крім того отримані результати можуть бути корисними при інтеграції даної системи в різні галузі.

SUMMARY

Robotic Garbage Sorting System.

The project contains 63 pages, 21 figures, 1 table, links to 22 literary sources annexes, 2 appendices and 4 design documents.

Keywords: ARTIFICIAL INTELLIGENCE, COMPUTER VISION, CONVOLUTIONAL NEURAL NETWORK, DEEP LEARNING, GARBAGE SORTING, MOBILE ROBOT, ROBOTICS, SYSTEM.

The object of development is a robotic garbage sorting system.

The goal of the project is to improve the accuracy of waste recognition by developing a classification model and the system that implements it using a convolutional neural network.

As part of this thesis project, a system was developed to classify various types of garbage — including batteries, organic, cardboard, clothing, glass, metal, paper, plastic, and shoes — in real time using webcam video streams and static images. To achieve this, the OpenCV library was used for real-time image processing from a laptop webcam. A software application was developed using the Python programming language, with a graphical user interface created using the Tkinter library. A convolutional neural network was built specifically for the purpose of garbage classification.

There is also potential for integrating this system into a mobile robot, enabling autonomous operation in real-world environments.

Moreover, the obtained results can be applied across various industries where automated garbage sorting and intelligent environmental monitoring are relevant.

Номер рядка	Формат	Позначення	Найменування	Кільк. аркушів	Номер екзем.	Примітка
1			Документація загальна			
2						
3			Знову розроблена			
4						
5	А4	ІК13.100БАК.006 ПЗ	Пояснювальна записка	63		
6	А3	ІК13.100БАК.006 Д1	Роботизована система сорту-	1		
7			вання сміття			
8			Схема алгоритму навчання			
9			мережі.			
10	А3	ІК13.100БАК.006 Д2	Роботизована система сорту-	1		
11			вання сміття			
12			Схема алгоритму роботи			
13			системи.			
14	А3	ІК13.100БАК.006 Д3	Роботизована система сорту-	1		
15			вання сміття			
16			Діаграма варіантів			
17			використання			
18	А3	ІК13.100БАК.006 Д4	Роботизована система сорту-	1		
19			вання сміття			
20			Діаграма послідовностей			
21						
22						
23						
24						
25						
26						
27						
28						

Зм.	Аркуш	№ докум.	Підпис	Дата	ІК13.100БАК.006 ТП		
Розробила		Килівник В.Л.			Роботизована система сортування сміття Відомість проєкту	Літ.	Аркуш 1 / Аркушів 1
Керівн.		Ткач М.М.				КПІ ім. Ігоря Сікорського	
Затв.							

Пояснювальна записка
до дипломного проєкту
на тему: «Роботизована система сортування сміття»

Київ – 2025 року

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ 4
ВСТУП 5
1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ 7
1.1 Проблема сортування відходів та сучасних технологічних рішень 7
1.2 Методи розв’язання задачі сортування сміття 10
Висновки до розділу 1 16
2 ФОРМУВАННЯ ВИМОГ ДО СИСТЕМИ 18
2.1 Функціональні вимоги 18
2.2 Нефункціональні вимоги 20
Висновки до розділу 2 22
3 ВИБІР НЕЙРОННОЇ МЕРЕЖІ 24
3.1 Обгрунтування та вибір нейронної мережі 24
3.2 Вибір і підготовка даних для навчання нейронної мережі 29
3.3 Вибір бібліотеки для реалізації системи сортування сміття 31
3.4 Вибір алгоритму оптимізації для навчання згорткової нейронної мережі . 31
Висновки до розділу 3 33
4 РЕАЛІЗАЦІЯ НЕЙРОННОЇ МЕРЕЖІ 35
4.1 Побудова моделі 35
4.2 Тренування моделі глибокого навчання 38
Висновки до розділу 4 43
5 ПРОЄКТУВАННЯ СИСТЕМИ ДЛЯ СОРТУВАННЯ СМІТТЯ 45
5.1 Вибір засобів для розробки 45
5.2 Реалізація роботизованої системи сортування сміття 47
5.2.1 Опис і тестування програмного додатку 48

5.2.2 Архітектурна здатність до розгортання на мобільному роботі 52
5.3 Результати роботи роботизованої системи сортування сміття 53
Висновки до розділу 5 59
ВИСНОВКИ 60
ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ 62
ДОДАТОК А 64

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

Adaptive Moment Estimation, Adam – алгоритм оптимізації.

Artificial intelligence, AI – штучний інтелект.

Computer Vision, CV – комп’ютерний зір.

Convolutional Neural Network, CNN – Згорткова нейронна мережа.

Internet of things, IoT – інтернет речей.

K-Nearest Neighbors, KNN – метод k-ближніх сусідів.

Machine learning, ML – машинне навчання.

Open Source Computer Vision Library, OpenCV – бібліотека алгоритмів комп’ютерного бачення.

Rectified Linear Unit, ReLU – функція активації.

Support vector machine, SVM – метод опорних векторів.

РНМ/(RNN) – Рекурентні нейронні мережі.

ШНМ – Штучні нейронні мережі.

ВСТУП

У XXI столітті стрімкий розвиток технологій автоматизації та робототехніки відкрив нові горизонти для розв’язання різноманітних прикладних задач. Особливу роль у цьому процесі відіграють мобільні роботи — автономні або напівавтономні пристрої, здатні переміщуватися у просторі, взаємодіяти з навколишнім середовищем і приймати рішення на основі отриманих даних. Сьогодні мобільні роботи активно застосовуються в логістиці, медицині, сільському господарстві, промисловості, рятувальних операціях, а також у побуті.

Завдяки поєднанню з інтелектуальними алгоритмами, такими як комп’ютерний зір і машинне навчання, мобільні роботи здатні виконувати складні задачі з високою точністю та адаптивністю. Один із головних напрямів розвитку це інтеграція комп’ютерного бачення, що дозволяє роботам «бачити» навколишній світ, розпізнавати об'єкти, орієнтуватися в просторі й ухвалювати рішення на основі зорової інформації. Такі технології значно розширюють можливості роботизованих систем у вирішенні задач, які раніше вимагали людської участі.

Серед актуальних проблем сучасності забруднення навколишнього середовища. Внаслідок стрімкого розвитку промисловості, транспорту та зростання населення, шкідливі викиди у воду та ґрунт та у повітря значно збільшився. Забруднення впливає не лише на екосистеми, але й на здоров’я людей, спричиняючи серйозні захворювання.

Серед основних причин забруднення – неконтрольовані викиди від промислових підприємств, вирубка лісів, накопичення побутових відходів та неправильна їх утилізація. Також значний внесок у погіршення екологічного стану робить використання пластика, який розкладається сотнями років.

Сортування сміття це надважлива роль, у тому щоб зберегти нашу планету. Багато відходів може бути направлена на переробку та використана повторно. Матеріали, такі як метал та скло, не втрачають свої властивості під час використання і можуть бути піддані переробці у багато разів. Не всі люди визнають і розуміють наскільки масштабна ця проблема, тому не сортують відходи і не

звертають на це достатньо уваги. Для вирішення даної проблеми існує рішення – це система сортування відходів за допомогою комп’ютерного зору. Сортування відходів це надважлива місія в різних сферах, як приклад, це такі сфери як робототехніка, автомобільна промисловість, сільське господарство, логістика та інше.

Наприклад, в робототехніці комп'ютерний зір для сортування відходів може використовуватись в автоматизованих системах збору сміття, а також в роботах, що працюють на сміттєсортувальних підприємствах. Роботи з комп'ютерним зором можуть автономно відбирати матеріали для переробки, наприклад, пластикові, скляні чи металеві вироби.

В автомобільній промисловості комп'ютерний зір використовується для сортування матеріалів, які підлягають переробці після того, як автомобілі вийшли з експлуатації. Це може включати металеві, пластикові та скляні деталі, а також гумові елементи.

В сільському господарстві комп'ютерний зір для сортування відходів може бути використаний для відділення органічних відходів від інших матеріалів, таких як пластик або метал. Наприклад, на фермах комп'ютерний зір може допомогти автоматизувати сортування харчових відходів або залишків сільськогосподарських культур.

Основне завдання даного проєкту – створити точну і надійну систему, яка зможе розпізнавати різні види мусору. У процесі виконання цього проєкту я розглянула одні з самих розповсюджених і популярних методів і алгоритмів сортування сміття. Ну і на основі цієї інформації була зроблена модель і навчена на великій кількості даних.

Крім того, у цьому проєкті розглянуто можливість розробки та впровадження системи сортування сміття з використанням комп’ютерного зору, яку потенційно можна інтегрувати в мобільного робота. Насамперед найбільшу увагу надається вибору методів комп’ютерного бачення, структури нейронної мережі, алгоритмів навчання та розробці архітектури системи, що дозволяє забезпечити її ефективну роботу в реальному середовищі.

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ

1.1 Проблема сортування відходів та сучасних технологічних рішень

Сортування побутових відходів це одна з найважливіших задач сучасного суспільства. Щороку людство продукує мільйони тонн сміття, значна частина якого потрапляє на полігони або забруднює довкілля, замість того щоб бути переробленою чи утилізованою. Проблема ускладнюється недостатньою ефективністю ручного сортування, низькою свідомістю населення та нестачею інфраструктури для повноцінного роздільного збору.

Традиційні методи сортування включають механічні системи з конвеєрами, магнітами, повітряними сепараторами, а також ручне сортування працівниками. Проте такі підходи є повільними, потребують значних ресурсів і є малоефективними при великому потоці змішаних відходів.

Метою даного проєкту є розробка системи, яка класифікує сміття для того, аби підвищити точність розпізнавання сміття. Для цього необхідно розробити модель класифікатора, який реалізується на базі згорткової нейронної мережі й системи. Особливістю задачі є необхідність розпізнавання різних типів сміття, які можуть бути схожі за зовнішніми ознаками.

Критерії ефективності системи сортування сміття є точність і швидкість. Якщо говорити про точність, то це обчислюється як відношення правильно ідентифікованих відходів до загальної кількості тестових зображень. Оскільки сортування сміття здійснюється в реальному часі з камери, з відео та із зображень, то можна стверджувати, що система має високу швидкість роботи.

Для цього застосовуються сучасні методи, такі як глибоке навчання, яке включає у себе згорткові нейронні мережі. Вони дозволяють з високою ефективністю аналізувати візуальну інформацію та навчатися на великому обсязі даних. Алгоритми глибокого навчання значно підвищує відсоток сортування і завдяки таким алгоритми як машинного навчання, нейронні мережі і комп'ютерний зір, система здатна аналізувати зображення і визначати їхній тип доволі точно.

Тому багато уваги акцентується на використанні інтелектуальних систем сортування, зокрема тих, що базуються на технологіях комп’ютерного бачення. В одному джерелі зазначається, що «Комп'ютерний зір — динамічна галузь на перетині інформатики та штучного інтелекту, наділяє машини здатністю інтерпретувати та розуміти візуальну інформацію з навколишнього світу.

Використовуючи складні алгоритми і передові технології обробки зображень, системи комп’ютерного зору дозволяють комп’ютерам витягувати значущу інформацію із зображень і відео, імітуючи можливості людського зору з надзвичайною точністю і ефективністю [1]».

Отже, Computer Vision використовує нейронні мережі, тому що вони навчаються на різноманітних наборах даних. Простіше кажучи, це дозволяє машинам так би мовити бачити зображення та відео, схоже на те, як це робить людина.

Комп’ютерний зір поєднує в собі як Artificial intelligence, AI так і machine learning, ML, тобто існує непреревний взаємозв’язок між цими галузями.

При виникненні запитання, яка різниця між людський баченням і комп’ютерним, є можливість переглянути таблицю 1.1, щоб побачити основні відмінності.

Таблиця 1.1 – Порівняння людського та комп’ютерного бачення

Критерій	Людське бачення	Комп’ютерне бачення
Обробка інформації	Інтуїтивна	Обчислювальна
Навчання	Від народження через досвід	Необхідне ML на великому Dataset
Точність	Доволі висока але залежить від таких ознак як втома, увага	Висока і стабільна
Швидкість	Миттєва	Залежить від систем і складності обробки
Освітлення	Хороша адаптація до умов	Залежить від якості зображення, освітлення

Тобто, можемо дійти до висновку, що все ж таки cv є потужним інструментом у багатьох галязях, але людське бачення і інтуїція є незамінними.

Як вже було сказано cv має безліч переваг, наприклад: точність, він може легко розпізнавати різноманітні об’єкти, наприклад, машини, товари, обличчя, відходи і таке інше.

Ще одна з переваг це витрати, так як система автоматизована то нема потреби в людській праці, що дуже сильно економить як час так і гроші для підприємств і не тільки.

Крім того, комп’ютерний зір дозволяє мобільному роботу самостійно аналізувати зображення з камери, приймати рішення без зовнішньої допомоги. Ще одна з переваг це реалізація в реальному часі, завдяки швидкій обробці зображень, робот здатен миттєво реагувати на наявність об’єктів для сортування.

І система може працювати цілодобово і без перерв, що дуже може бути корисним наприклад у відеонагляді, систем безпеки і таке інше.

Також, комп’ютерне бачення можливо ефективно використовувати з AI, ML і IoT, це підвищить ефективність систем. Звичайно, що це не всі переваги CV, але це, як на мою думку, самі основні.

Хоча ми побачили доволі багато переваг комп’ютерного бачення, як завжди є і недоліки, а саме:

Це обладнання, ті самі камери, потужні комп’ютери, спеціальне software та інше може бути дорогим. Крім того, якщо камера не якісно знімає, то можуть виникати помилки в класифікації або взагалі може не впізнати чи переплутати предмет. І мобільні роботи мають обмежену обчислювальну потужність, що може впливати на швидкість розпізнавання.

Також зовнішні фактори, такі як зміни в освітленні, погодні умови, тіні або перешкоди можуть суттєво знижувати точність розпізнавання об’єктів.

Для коректного розпізнавання зображень необхідний великий Dataset на певну тему, на створення якого йде від сотень до навіть мільйонів прикладів зображень. Це доволі клопіткий процес. До того ж, необхідно навчити модель і систему вірно працювати, це може визивати складнощі у налаштуванні.

1.2 Методи розв’язання задачі сортування сміття

Класичні методи машинного навчання: алгоритми, які використовують для прогнозування та як вже було сказано раніше, машинне навчання і глибоке навчання дають можливість системам розпізнавати і класифікувати різні види сміття на основі таких алгоритмів як форма, розмір і колір. Одні з найпоширеніших алгоритмів це:

Група методів машинного навчання з учителем, які застосовуються для задач класифікації, регресії, а також виявлення аномалій це SVM. Метод базується на побудові оптимальної гіперплощини, що розділяє дані різних класів у багатовимірному просторі [2].

Цей алгоритм дуже ефективний при роботі з великими даними і зберігає точність попри те, що кількість ознак перевищує кількість навчальних прикладів. Також, цей алгоритм дуже гнучний у налаштуванні. Можливо використовувати ядрові функції для побудови гіперплощини. Важливо вірно підібрати ядро і параметри регуляризації, якщо ознак більше ніж зразків, інакше можливо перенавчання.

Також простий алгоритм у реалізації машинного навчання з учителем це KNN. Його використовують для задач регресії, класифікації. Він ґрунтується на ідеї, що спостереження, найближчі до заданої точки даних, є найбільш «схожими» спостереженнями в наборі даних, і тому ми можемо класифікувати непередбачені точки на основі значень найближчих існуючих точок [3].

Хоча цей алгоритм має багато переваг, а саме, простота реалізації, ефективний при достатній кількості даних, гнучкий для задач регресії і класифікації, але має високу обчислювальну складність якщо великі об’єми даних, залежний від масштабування ознак і працює погано з високовимірними даними.

Ще один потужний і надійний алгоритм це Random Forest, випадковий ліс. Його застосовують часто для класифікації та регресії. Як видно з самої назви, його суть полягає в поєднанні великої кількості дерев рішень, які працюють разом для прийняття остаточного рішення. Такий підхід належить до групи ансамблевого навчання, що дозволяє досягти більшої точності та стабільності результатів [4].

Розглянемо приклад для розуміння принципу роботи цього алгоритму, ви просите поради в колі сім’ї та родичів щодо вибору місця для роботи. Кожна людина пропонує свій варіант, на основі власного розуміння і уподобань, як дерево рішення, навчане на різних даних. Остаточне рішення ви приймаєте ви дивлячись на думку більшості або усереднюючи відповіді — саме за таким принципом працює Random Forest.

Як ми побачили такі алгоритми як Support Vector Machine, SVM, Random Forest, випадковий ліс та алгоритми найближчих сусідів, k-NN застосовуються для класифікації, але ефективність цих методів знижується по різним причинам і також, коли відбувається робота з не якісними зображеннями.

Семантична сегментація зображень*:* завдання комп’ютерного зору, яке передбачає призначення мітки кожному пікселю зображення називається семантичною сегментацією. Цю техніку можна використовувати в різних сферах, таких як автономне водіння, розпізнавання об’єктів і медична візуалізація [5].

Семантична сегментація зображень вирізняється високим рівнем деталізації та точністю, оскільки кожному пікселю на зображенні присвоюється мітка певного класу. Це дозволяє моделі не лише розпізнати, які об'єкти присутні на зображенні, а й визначити їх точне розташування.

На відміну від звичайної класифікації чи об’єктної детекції, семантична сегментація охоплює всю площину зображення, включаючи фонові області, що робить її надзвичайно корисною для задач, де важлива повна картина сцени. Проте вона не розрізняє окремі об'єкти одного класу: всі, наприклад, дерева чи автомобілі, маркуються однаково.

На рисунку 1.2 можемо побачити приклад семантичної сегментації зображення. Голубим кольором позначено фон, зеленим кольором позначено основний об’єкт, а рожевим кольором може вказувати на особливу зону переходу між фоном та об’єктом, або пошкоджену ділянку.

Такий тип сегментації дозволяє машині "розуміти", які частини зображення відповідають окремим семантичним класам. Це є ключовим етапом для задач

комп’ютерного бачення, наприклад, аналіз зображень, розпізнавання об’єктів, автономна навігація та інше.

Тобто, семантична сегментація не просто класифікує об’єкти, але і визначає їх точне положення, але вона потребує значних обчислювальних ресурсів і дуже точно розмічених зображень, щоб кожен піксель мав мітку.

Наприклад, всі пікселі, що належать до класу "пластик", будуть позначені як пластик, хоча вони належать до різних об'єктів або контекстів. Це відрізняється від об'єктної сегментації, де кожен об'єкт на зображенні має бути окремо розпізнаний.



Рисунок 1.2 – Приклад семантичної сегментації зображення

Існує багато методів семантичної сегментації, розглянемо де-які з них:

Однією з найпопулярніших моделей є U-Net, бо вона використовує конволюційні нейронні мережі для того, щоб виділити ознаки а потім об'єднує їх, і так виходить високоточна сегментація.

Теж один з найпопулярніших підходів для семантичної сегментації це DeepLab розроблених Google Research, тому що використовує глибокі конволю-

ційні нейронні мережі та різні техніки такі як, ділітація. Він може фіксувати деталі і виконувати семантичну сегментацію зображень з доволі хорошою точністю.

Сематична сегментація має такі переваги, як точність, тому що можна точно класифікувати об'єкт і локалізувати його, крім того є гнучким, тому що можна використовувати для різних типів об'єктів. Але вимагає дуже якісних даних, щоб була висока точність сегментації та необхідний потужне обладнення з великим об'ємом даних.

Нейронні мережі*:* потужним інструментом у сфері машинного навчання є нейронні мережі, які основані на імітації роботи мозку людини. Вони складаються з великої кількості взаємопов’язаних елементів — нейронів, які організовані в різні шари.

До складу нейронної мережі належать такі компоненти: вхідний шар, приховані шари, вихідний шар, функція активації, вага та зсув. Інформація потрапляє до вхідного шару, де нейрони обробляють дані, потім визначають їх категорії чи аналізують їх. Потім ці дані передаються через з'єднання на наступний шар. Кожне з'єднання має свою вагу, а кожен нейрон в наступному шарі може отримувати кілька вхідних сигналів. Далі дані проходять через приховані шари, яких може бути багато. В кожному з них нейрони з'єднані між собою, і кожен новий шар обробляє результат роботи попереднього. Інформація передається далі через всі шари, поки не досягає вихідного шару.

Для навчання такої мережі необхідні дані, які можуть бути розміченими і не розміченими. Вірно сказати, що процес навчання моделі – виконання якогось визначеного завдання. Вона дізнається, які зв’язки між нейронами важливі для успішного прогнозування, далі на кожному етапі алгоритм використовує математичну функцію для визначення різниці між передбаченням та очікуваним результатом — це називається помилкою. Цю помилку система використовує аби розрахувати те, як потрібно змінити ваги нейронів, щоб покращити точність прогнозів. Після численних циклів навчання та налаштування алгоритм поступово стає здатним робити точніші прогнози. Це триває, поки нейронна мережа не досягне максимальної ефективності.

Нейронні мережі можна використовувати у різних сферах і обтяжливих процесів, щоб створити автономні транспортні засобі, генерацію нового контенту, обробку медичних зображень для діагностики хвороб, класифікація та сегментація зображень і для поставленої задачі сортування сміття за допомогою комп'ютерного зору, бо вони здатні розпізнавати різні типи відходів на зображеннях, наприклад, пластик, папір, метал, скло та інше. Мережа може навчатися на основі даних без явних вказівок та обробляють великі кількість даних одночасно.

Навчання без учителя: цей метод навчання, застосовується тоді, коли дані не мають міток або правильних відповідей. Модель намагається знайти закономірності, структури чи групи у даних без явних вказівок. Метою такого навчання є виявлення прихованих структур, закономірностей або групувань у даних без попереднього знання того, що саме потрібно знайти. Один із найпоширеніших методів це кластеризація, коли алгоритм групує схожі між собою об'єкти. Ще один приклад це зниження розмірності, яке дозволяє спростити уявлення про складні дані без втрати важливої інформації.

У контексті зображень навчання без учителя може застосовуватись, наприклад, для виявлення типових шаблонів чи структур у великій колекції зображень, де відсутня ручна розмітка. Це особливо корисно, коли маркування даних є надто дорогим або практично неможливим.

Проте такий підхід зазвичай не дає настільки чітких або контрольованих результатів, як навчання з учителем, оскільки модель не має конкретних орієнтирів для порівняння. Тим не менш, навчання без учителя часто використовується як попередній етап для подальшого аналізу або у комбінації з іншими методами, зокрема у напівавтоматичних або самонавчальних системах.

Навчання з учителем: Цей тип навчання передбачає що є розмічені дані, які використовуються потім для тренування моделі. Кожен приклад у навчальному наборі містить вхідні дані і відповідну правильну відповідь, яка називається міткою, яку модель повинна передбачити. Тобто для такого навчання потрібен набір даних,

де кожне зображення матиме відповідну мітку. Під час навчання алгоритм поступово знаходить залежності між вхідними характеристиками та мітками, щоб надалі робити точні передбачення на нових, нерозмічених даних.

У випадку семантичної сегментації, навчання з учителем передбачає використання зображень разом із точними піксельними мітками, що дозволяє моделі вивчити, як розпізнавати й класифікувати кожну область на зображенні. Цей підхід забезпечує високу точність, але потребує великого обсягу якісно розмічених даних, що є трудомістким і затратним процесом.

Саме це навчання використовується у цьому дипломному проєкті, оскільки дані структуровані у вигляді папок, де назва папки вказує на тип сміття, а це фактично є мітками.

Навчання з частковим залученням учителя: Цей підхід поєднує переваги навчання з учителем і без нього. У таких задачах використовується невелика кількість розмічених даних разом із великою кількістю нерозмічених. Таке навчання використовується для розв’язку складних задач. Принцип роботи такий: модель спочатку навчається на розмічених прикладах.

Потім вона поширює знання на нерозмічені дані — наприклад, передбачає мітки, класи чи ознаки. Далі вона може вдосконалюватися, використовуючи як розмічені, так і передбачені дані. Навчання з учителем є компромісом між двома іншими методами і дозволяє зменшити залежність від великої кількості розмічених даних.

Наприклад є задача класифікації зображень із 100 000 не розмічених зображень тільки 1 000 мають мітки, наприклад "помідор", "картопля", а решта — ні. Алгоритм вчиться на цих 1 000 мітках, а потім використовує нерозмічені зображення, щоб уточнити гіпотези про структуру даних.

Такий підхід особливо корисний у сферах, де розмічені дані є обмеженими або дорогими у створенні наприклад, у медичній діагностиці, розпізнаванні об'єктів, автоматичному сортуванні сміття або аналізі супутникових зображень.

Навчання з частковим залученням учителя допомагає моделі краще узагальнювати, використовуючи великий обсяг даних, які інакше залишилися б незадіяними. Це робить його ефективним компромісом між точністю повністю контрольованого навчання і гнучкістю безконтрольних методів.

Висновки до розділу 1

У результаті опрацювання матеріалів першого розділу було проведено глибокий аналіз проблеми сортування відходів та сучасних підходів до її автоматизації. Розглянуто як традиційні методи, так і новітні інструменти комп'ютерного зору та машинного навчання, зокрема — глибокого навчання, що відкривають нові можливості для реалізації ефективних, швидких та масштабованих рішень.

Детально проаналізовано класичні методи машинного навчання, які застосовуються для задач класифікації та розпізнавання об'єктів. Було розглянуто такі популярні алгоритми як SVM, k-NN та Random Forest, які показують добру ефективність у задачах класифікації. Водночас, підкреслено, що ці методи можуть мати обмеження при роботі з неякісними або складними зображеннями, а також в умовах високої розмірності даних.

Окрему розглянули метод семантичною сегментації зображень — техніці комп'ютерного зору, яка забезпечує високу точність та деталізацію при класифікації кожного пікселя зображення. Цей підхід особливо корисний у задачах, де важлива повна інформація про сцену. Розглянуто сучасні архітектури, зокрема U-Net і DeepLab, які є провідними рішеннями для виконання семантичної сегментації у різних галузях, включно з сортуванням сміття.

Також було описано роль нейронних мереж та особливості їх побудови й навчання. Згорткові нейронні мережі (CNN) є основою для більшості сучасних систем візуального розпізнавання, зокрема й у сфері сортування сміття. Завдяки здатності аналізувати складні візуальні дані, CNN дозволяють моделі ефективно відрізняти типи відходів, навіть за умов високої варіативності візуальних характеристик.

Проаналізовано три основні підходи до машинного навчання: навчання з учителем, що передбачає використання розмічених даних та забезпечує високу точність, але потребує значних ресурсів для збору даних, навчання без учителя, яке дозволяє аналізувати великі обсяги нерозмічених даних, але не дає такого ж рівня контролю та навчання з частковим залученням учителя, яке поєднує переваги обох підходів, зменшуючи залежність від великої кількості вручну розмічених даних.

У рамках дипломного проєкту обґрунтовано доцільність використання підходу навчання з учителем з використанням розміченого датасету, в якому структура даних (папки з назвами типів сміття) дозволяє ефективно тренувати модель для задачі класифікації.

2 ФОРМУВАННЯ ВИМОГ ДО СИСТЕМИ

2.1 Функціональні вимоги

Розроблювана роботизована система призначена для автоматичної класифікації об'єктів сміття з використанням методів комп’ютерного зору. Її створення вимагає чіткого визначення вимог, які формують основу для реалізації ключових функціональних і технічних компонентів. У цьому розділі наведено перелік функціональних і нефункціональних вимог, сформульованих на основі аналізу предметної області, огляду подібних рішень та технічних особливостей задачі.

Функціональні вимоги окреслюють основні можливості системи, що забезпечують її практичну цінність. Вони визначають, які функції має реалізовувати програмне забезпечення, а також як користувач взаємодіє з інтерфейсом для виконання необхідних дій.

Система повинна мати здатність розпізнавати об’єкти на зображеннях або у відеопотоці в реальному часі. Розпізнавання здійснюється за допомогою попередньо навченої згорткової нейронної мережі, яка класифікує об’єкти на один із дев’яти встановлених класів: батарейки, біологічні відходи, коробки, одяг, скло, метал, папір, пластик та взуття. Такий підхід дозволяє забезпечити точне сортування матеріалів для подальшої обробки чи утилізації.

Система підтримує кілька типів введення даних: завантаження зображень із локального носія, відкриття відеофайлів та використання камери в режимі реального часу. Це забезпечує її можливість адаптуватися до різних сценаріїв використання, тому це її робить універсальною.

Важливим елементом функціональності є наочне відображення результатів класифікації. Після обробки зображення або відеокадру, на екрані користувача повинна з’являтися назва визначеного класу сміття разом із показником ймовірності розпізнавання. Усі результати відображаються в інтерактивному графічному інтерфейсі, що забезпечує зручність взаємодії навіть для користувачів із мінімальним технічним досвідом.

Інтерфейс реалізовано з використанням бібліотеки Tkinter і представлено у вигляді настільного застосунку. Користувач має доступ до трьох основних дій: завантаження зображення, відкриття відеофайлу та запуск камери. Кожна дія активується натисканням відповідної кнопки. Після обробки система відображає вхідне зображення або відео з нанесеним підписом класифікованого об'єкта та рівнем впевненості моделі. Такий підхід дозволяє зробити процес сортування сміття максимально зрозумілим і доступним навіть для користувачів без технічного досвіду.

Для стабільної роботи моделі всі вхідні зображення автоматично масштабуються до фіксованого розміру 224×224 пікселів — оптимального для обробки нейронною мережею.

Передбачено також можливість використання камери, встановленої на мобільного робота, що дозволяє адаптувати систему для автономного сортування сміття на рухомій платформі в перспективі практичного застосування.

Система повинна підтримувати адаптивну реакцію на зміну освітлення або якості вхідних зображень, що є критичним для реального середовища експлуатації, де умови зйомки не завжди передбачувані. Це можливо завдяки використанню попередньої нормалізації пікселів і багаторівневої фільтрації ознак на рівні згорткових шарів моделі.

Також необхідно забезпечити швидкий час відгуку при класифікації зображень, що надходять із камери або відеофайлів. Затримка між надходженням кадру і відображенням результату не повинна перевищувати декількох сотень мілісекунд. Це є важливою умовою при використанні системи в динамічних сценаріях, зокрема на рухомих сортувальних лініях або автономних платформах.

Крім того, система повинна бути масштабованою та відкритою до подальшого вдосконалення. Це передбачає можливість навчання моделі на нових класах відходів, інтеграцію з базами даних чи платформами керування утилізацією, а також розширення функціоналу для збереження статистики, формування звітів та ведення логів класифікацій.

2.2 Нефункціональні вимоги

Нефункціональні вимоги визначають якісні характеристики системи, які мають вплив на її результативність, використовуваність та стабільність роботи. Однією з ключових вимог є забезпечення точності класифікації не нижче 60% на основі наявного навчального набору даних. Це значення встановлює базовий рівень ефективності, необхідний для практичного застосування системи.

Система повинна демонструвати високу надійність і стабільність функціонування, безперервно працюючи протягом тривалого часу без збоїв, помилок або зниження продуктивності — незалежно від типу джерела вхідних даних (зображення, відео тощо). Це особливо важливо в умовах інтенсивної експлуатації, наприклад, у промислових чи міських сортувальних центрах.

Надійність та стабільність: система повинна забезпечувати стабільну роботу під час тривалого використання. Це особливо важливо в режимі реального часу, коли камера передає безперервний потік кадрів. Модель повинна бути здатна класифікувати зображення без збоїв або аварійного завершення роботи. Програма повинна обробляти типові помилки наприклад, відсутність відео, помилки читання файлів тощо і забезпечувати відповідне інформування користувача.

Продуктивність: час обробки одного кадру зображення не повинен перевищувати 200–300 мілісекунд, що дозволяє здійснювати класифікацію у майже реальному часі. Це досягається за рахунок ефективної структури згорткової нейронної мережі, оптимізації коду та використання попередньо масштабованих вхідних даних. Модель навчається лише один раз, після чого використовується виключно для прогнозування, що мінімізує навантаження на систему під час експлуатації.

Портативність і сумісність: система має бути сумісною з основними операційними системами, що підтримують бібліотеки Python, наприклад, Windows, Linux. Програмна реалізація повинна працювати на середньостатистичних комп’ютерах без потреби у графічних процесорах, хоча наявність GPU може прискорити роботу моделі.

Зрозумілість та простота використання: користувацький інтерфейс має бути інтуїтивно зрозумілим, з мінімально необхідною кількістю дій для виконання кожної функції. Графічні елементи оформлені в єдиному стилі, а написи і кнопки мають зрозумілі назви українською мовою. Це дозволяє ефективно користуватися системою навіть тим, хто не має досвіду роботи з подібним програмним забезпеченням.

Безпека: система повинна гарантувати безпечну обробку локальних файлів без передачі даних у мережу. Жодна інформація з камер або локальних носіїв не зберігається автоматично й не передається третім сторонам. Також варто уникати виконання стороннього коду, що може загрожувати безпеці користувача під час запуску програми.

Масштабованість та можливість розширення: архітектура системи повинна бути доступною до можливості подальшого розширення функціональності без повного перепроєктування. Це включає можливість додавання нових класів сміття, підключення баз даних, інтеграцію з системами управління утилізацією, а також використання нових моделей комп'ютерного зору. У перспективі система може бути адаптована до використання в мобільних застосунках або на роботизованих платформах для автоматичного сортування сміття.

Документованість: код програми має бути задокументованим належним чином. Усі основні функції та класи повинні супроводжуватися коментарями, що описують їхнє призначення. Наявність документації спрощує модифікацію та підтримку системи в майбутньому, а також дає можливість іншим розробникам швидко ознайомитись із логікою роботи проєкту.

Завдяки універсальній архітектурі програмного забезпечення, система потенційно може бути адаптована для роботи на мобільних роботизованих платформах, оснащених камерами. Це відкриває широкі перспективи для застосування у сфері автономного збирання та сортування відходів, особливо у віддалених або важкодоступних локаціях.

У підсумку, сформульовані нефункціональні вимоги становлять основу для побудови сучасної, ефективної та технологічно гнучкої системи автоматизованої

класифікації сміття, що має прикладну цінність для екологічного управління та сталого розвитку.

Висновки до розділу 2

У результаті аналізу було детально сформульовано функціональні та нефункціональні вимоги, які є основою для створення ефективної системи автоматизованого сортування сміття на основі методів комп’ютерного зору та глибокого навчання. Ці вимоги дозволяють не лише окреслити технічні параметри майбутньої розробки, а й гарантувати її практичну реалізацію та інтеграцію в реальні умови експлуатації.

Функціональні вимоги визначають набір основних можливостей, які забезпечують виконання ключових завдань системи: автоматичну класифікацію об'єктів сміття на дев’ять попередньо визначених класів, обробку зображень і відео з різних джерел (локальних, потокових, з камери), виведення результатів із ймовірнісною оцінкою та інтерактивний інтерфейс користувача. Особливу увагу приділено забезпеченню зручності взаємодії користувача із системою, навіть за мінімального рівня технічної підготовки, що робить її доступною для широкого кола потенційних користувачів — від пересічних громадян до операторів сортувальних ліній.

Наявність підтримки роботи в реальному часі, можливість обробки даних з мобільної камери, а також реалізація масштабування інтерфейсу свідчать про високий рівень універсальності системи. Це дозволяє адаптувати її до різних сценаріїв — від використання в побуті до впровадження на підприємствах із сортування та переробки відходів.

Нефункціональні вимоги охоплюють якісні характеристики системи, які мають визначальне значення для її довготривалого та надійного функціонування. Серед них — забезпечення стабільної та безперебійної роботи, швидкої обробки даних, сумісність із популярними операційними системами, безпечна робота з файлами без потреби передачі даних у мережу, а також відповідність вимогам

безпеки і конфіденційності. Урахування цих факторів є вкрай важливим, особливо в умовах промислової експлуатації чи інтеграції з автоматизованими роботизованими платформами.

Окремо варто відзначити вимоги до масштабованості та розширюваності, які відкривають можливість для подальшого розвитку системи. Це включає оновлення або донавчання моделі на нових класах відходів, підключення до централізованих баз даних, розширення функціоналу для аналітики, формування звітності чи впровадження телеметрії, тобто система не є статичною — її архітектура враховує потенціал до зростання та еволюції разом із технологічними змінами та потребами суспільства.

Отже, сформовані вимоги не лише забезпечують структуровану основу для подальшої реалізації програмного забезпечення, але й дають змогу впевнено стверджувати про високу прикладну цінність проєкту. У перспективі, така система може стати частиною ширших екосистем з управління відходами, внести вклад у цифрову трансформацію екологічного сектору та сприяти підвищенню рівня автоматизації, екологічної свідомості та відповідального поводження з ресурсами..

3 ВИБІР НЕЙРОННОЇ МЕРЕЖІ

3.1 Обгрунтування та вибір нейронної мережі

У сучасних задачах класифікації зображень дедалі частіше застосовуються нейронні мережі, зокрема штучні нейронні мережі, ШНМ. Вони є ефективним інструментом для вирішення задач, де необхідно автоматично віднести зображення до одного з попередньо визначених класів. Такий підхід демонструє високу точність і адаптивність навіть у випадках складних або неоднозначних вхідних даних.

Штучні нейронні мережі: це один із ключових інструментів машинного навчання. Вони складаються з великої кількості взаємопов'язаних елементів — штучних нейронів або вузлів, які здатні навчатися на основі даних, виявляти закономірності та здійснювати прогнозування або класифікацію. На рисунку 3.1 представлено візуалізацію структури штучної нейронної мережі.

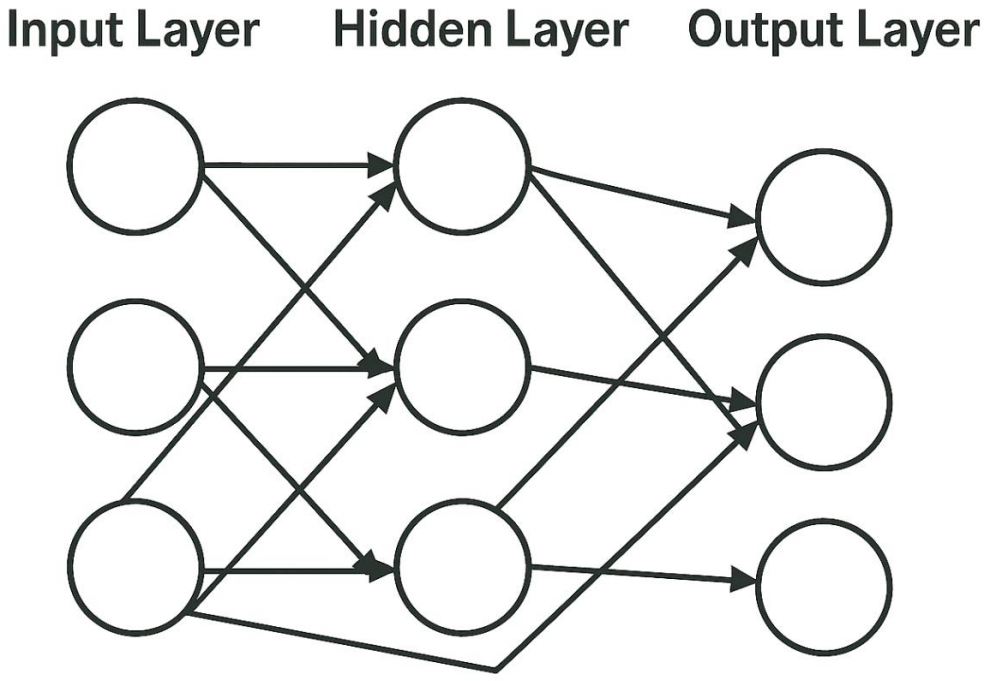


Рисунок 3.1 – Структура штучної нейронної мережі

Кожен нейрон у мережі приймає вхідні дані (наприклад, числові значення), обробляє їх із застосуванням певних вагових коефіцієнтів та функції активації, а

потім передає обчислений результат на наступний рівень. Усі нейрони згруповані в шари: вхідний шар приймає первинну інформацію, декілька прихованих шарів виконують внутрішню обробку, а вихідний шар генерує остаточний результат — наприклад, клас, до якого відноситься об’єкт на зображенні.

Будь-яка мережа, побудована з таких взаємопов’язаних структур, є штучною нейронною мережею, здатною ефективно вирішувати складні завдання аналізу зображень.

Для навчання штучних нейронних мереж зазвичай використовують два основних підходи: навчання з учителем та навчання без учителя.

Як зазначалося раніше, навчання з учителем передбачає наявність заздалегідь підготовлених розмічених даних, де відомий правильний результат. На основі цих прикладів модель поступово навчається здійснювати класифікацію чи прогнозування. Натомість, під час навчання без учителя модель самостійно аналізує вхідні дані, виявляє приховані закономірності та встановлює взаємозв’язки без попередньо заданих міток або відповідей.

Рекуретні нейронні мережі*:* це нейронні мережі RNN, які навчені на послідовних або часових рядах даних для створення моделі ML, яка може робити висновки на основі послідовних вхідних даних [6]. На рисунку 3.2 можемо побачити рекуретну мережу в згортці.

Ключовою особливістю RNN є наявність зворотного зв’язку: вихід моделі на попередньому часовому кроці подається назад як частина вхідних даних для наступного кроку. Завдяки цьому мережа здатна "запам’ятовувати" попередні стани та враховувати контекст у часі.

На відміну від класичних нейронних мереж, RNN володіють здатністю зберігати інформацію про попередні вхідні значення, що дозволяє їм впливати на обробку поточних даних. Іншими словами, результат, який видає мережа, залежить не лише від поточного входу, але й від усієї попередньої історії вхідних сигналів.

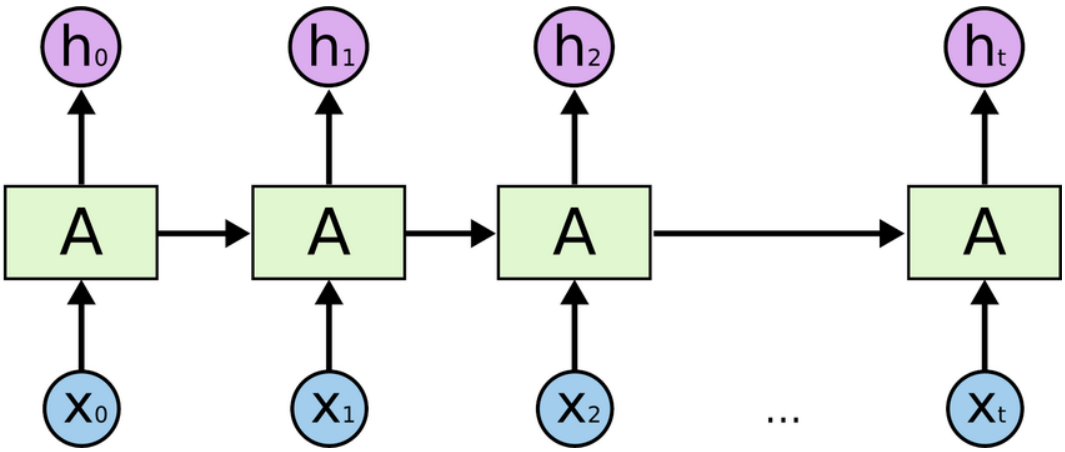


Рисунок 3.2 – Рекурентна мережа в розгортці [7]

Згорткові нейронні мережі: це тип нейронних мереж, які найчастіше використовуються для обробки зображень, відео, класифікації об’єктів на фотографіях тощо. CNN є фундаментом сучасного комп’ютерного зору, оскільки здатні вирішувати широкий спектр завдань — від базової класифікації до виявлення та сегментації об’єктів.

Згорткова нейронна мережа за рахунок застосування спеціальної операції – згортки, дозволяє водночас зменшити кількість інформації, що зберігається в пам'яті інформації, за рахунок чого краще справляється з картинками більш високої роздільної здатності, і виділити опорні ознаки зображення, такі як ребра, контури або грані. На наступному рівні обробки з цих ребер і граней можна розпізнати повторювані фрагменти текстур, які далі можуть скластися в фрагменти зображення [8].

Однією з ключових переваг згорткових мереж є те, що вони автоматично виявляють ознаки, features, на зображеннях, на відміну від класичних методів, де інженери вручну визначали необхідні характеристики. CNN складаються з декількох типів спеціалізованих шарів, які працюють разом для виділення та інтерпретації візуальної інформації. Типові шари згорткової нейронної мережі:

Згортковий шар, Convolutional Layer, Агрегувальний шар, Max Pooling Average Pooling, Функція Активації, Rectified Linear Unit, Шар підвибірки Pooling Layer, Повнозв’язний шар, Fully Connected Layer, шар втрат, Loss Layer.

Одним із класичних прикладів застосування CNN є 3D-візуалізація згорткової нейронної мережі, яка розпізнає рукописні цифри, наприклад із набору даних MNIST [9]. Така мережа проходить кілька рівнів перетворення, поступово переходячи від сирого піксельного зображення до впевненого визначення цифри. На рисунку 3.3 зображена візуалізація CNN.

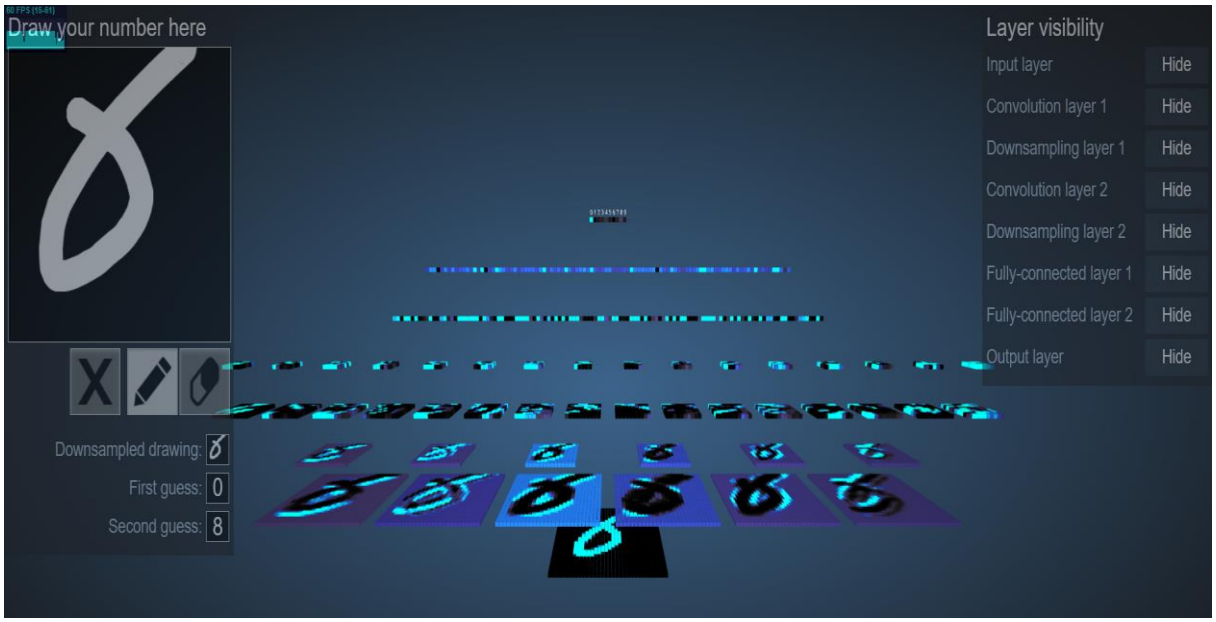


Рисунок 3.3 – 3D візуалізація Convolutional Neural Network

Цей підхід є одним із найкращих та найефективніших для задач, пов’язаних із комп’ютерним зором. Саме тому було прийнято рішення використовувати згорткову нейронну мережу як основну модель для розв’язання поставленої задачі.

Алгоритм YOLO, You Only Look Once*:* якщо говорити про сучасні алгоритми комп’ютерного зору, зокрема архітектури, побудовані на основі CNN, то однією з найефективніших і найпопулярніших є YOLO. Цей алгоритм призначений для детекції об’єктів на зображеннях у реальному часі.

Основна особливість YOLO це те, що він обробляє все зображення в один прохід, на відміну від традиційних підходів, де аналіз здійснюється послідовно через сканування зображення невеликими фрагментами (вікнами). Завдяки цьому підходу суттєво зменшується час обробки, що робить YOLO надзвичайно ефективним у задачах реального часу, наприклад: відеоспостереженні, автономному водінні або в робототехніці.

YOLO виконує детекцію об'єктів шляхом поділу зображення на сітку і передбачення ймовірності наявності об’єкта та його координат у кожній частині цієї сітки. Такий підхід дозволяє моделі не лише розпізнавати об’єкти, а й точно визначати їхнє розташування.

З моменту появи першої версії YOLO було розроблено низку оновлень та модифікацій — YOLOv2, YOLOv3, YOLOv4, YOLOv5, YOLOv6 та новітні варіанти, такі як YOLOv7, YOLOv8, а також полегшені моделі для роботи на пристроях з обмеженими ресурсами. На рисунку 3.4 зображено порівняння моделей YoloV.

Кожна наступна версія демонструє покращення точності, швидкості та гнучкості, дозволяючи адаптувати алгоритм до різноманітних задач комп’ютерного зору, зокрема: виявлення об'єктів у складних умовах освітлення; мультикласова класифікація; сегментація об'єктів; трекінг у відеопотоці.

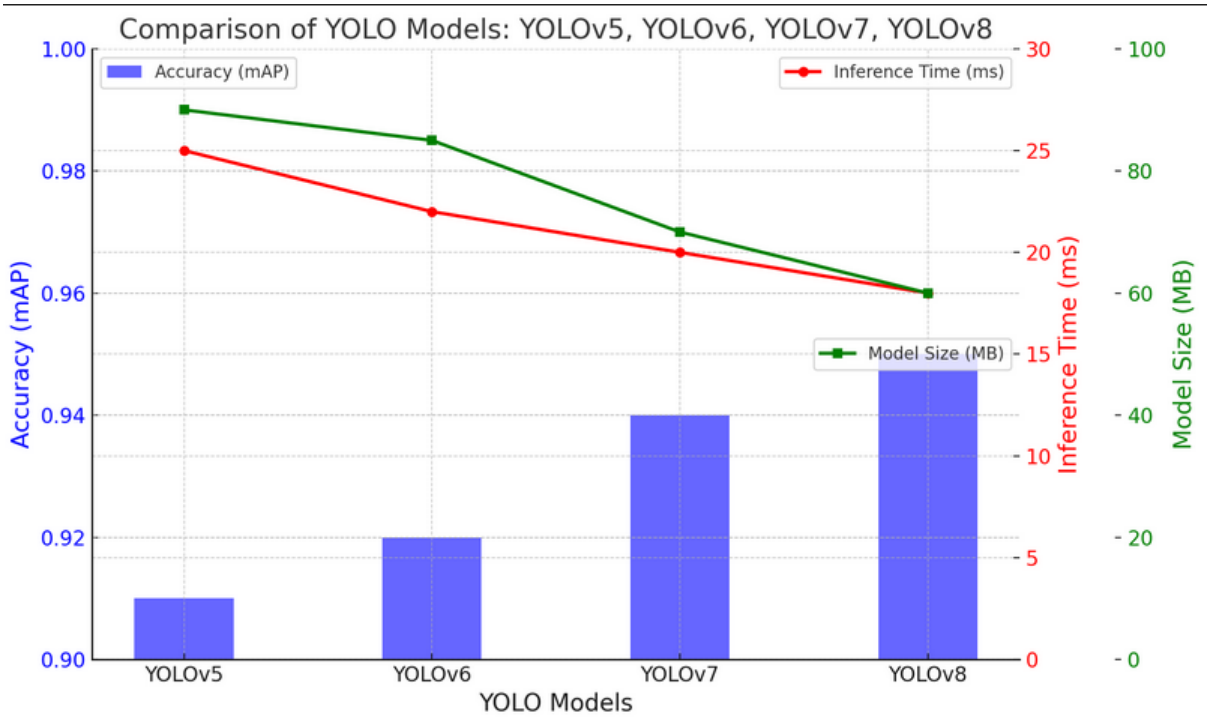


Рисунок 3.4 – Порівняння моделей YOLOv

Основна ідея YOLO зводиться до того, що модель розглядає все зображення одразу, як єдине ціле, на відміну від інших методів, які сканують окремі ділянки

зображення послідовно. Завдяки цьому досягається цілісне сприйняття контексту, що дозволяє краще враховувати взаємозв’язки між об’єктами на сцені.

На першому етапі обробки YOLO поділяє вхідне зображення на сітку розміром S × S. Кожна клітинка цієї сітки відповідає за виявлення об'єкта, центр якого потрапляє в межі цієї клітинки.

3.2 Вибір і підготовка даних для навчання нейронної мережі

Одним із ключових етапів розробки системи автоматизованого сортування сміття є вибір якісного та релевантного набору даних для навчання нейронної мережі. Відповідний набір повинен містити різноманітні зображення об’єктів, що належать до типових категорій відходів: пластик, папір, скло, метал, органічні рештки та інше.

У процесі підбору датасету було проаналізовано кілька відкритих джерел:

TrashNet — набір з 6 класами (пластик, метал, папір, картон, скло, загальне сміття), який містить приблизно 2500 зображень. Незважаючи на просту структуру, цей датасет є обмеженим як за кількістю класів, так і за обсягом даних, що ускладнює ефективне навчання сучасних моделей глибокого навчання.

TACO (Trash Annotations in Context) — набір з понад 60 класами та великою кількістю зображень. Хоча він має хороше покриття реальних сценаріїв, деякі об'єкти є візуально схожими, а сміття на багатьох зображеннях частково приховане або розмите. Такі особливості можуть ускладнити навчання моделі та вплинути на її точність.

У результаті проведеного аналізу було прийнято рішення в користь використання датасету "Garbage Classification" [10], як основного. Цей набір даних містить близько 20 000 зображень, розділених на 9 категорій: battery, biological, cardboard, clothes, glass, metal, paper, plastic, shoes

Датасет має збалансовану структуру, проте деякі класи представлені меншою кількістю прикладів, що може призвести до дисбалансу класів під час навчання. Щоб покращити якість моделі та зробити розподіл класів більш рівномірним, було здійснено розширення основного набору даних шляхом інтеграції додаткових

зображень з інших джерел: Garbage Classification [11], Classification_Garbage [12], Food Images (Food-101) [13], garbage_classification_splitted [14].

У результаті об’єднання всіх джерел сформовано загальний датасет, який налічує приблизно 35 000 зображень. Це дає можливість кардинально покращити якість навчання нейронної мережі та підвищити її загальну здатність до узагальнення на нових, не бачених раніше прикладах. На рисунку 3.5 можемо побачити приклади вхідних данних.



Рисунок 3.5 – Приклади вхідних даних

3.3 Вибір бібліотеки для реалізації системи сортування сміття

Мій вибір зупинився на OpenCV, скорочення від Open Source Computer Vision Library, тому що це безкоштовна бібліотека для комп’ютерного зору та машинного навчання з відкритим кодом. Її розробила компанія Intel у 2000 році [15] і відтоді стала одним із найпопулярніших інструментів у сфері обробки зображень.

У задачі сортування сміття за допомогою комп’ютерного зору бібліотека OpenCV відіграє важливу роль, оскільки забезпечує виконання основних функцій — зчитування зображень, їх попередню обробку та розпізнавання об’єктів. За допомогою OpenCV можливо зчитувати зображення з фотографій або відео, здійснювати обробку відеопотоку в реальному часі, виявляти та ідентифікувати об'єкти наприклад, обличчя, автомобілі, предмети побуту, розпізнавати форму, колір та інші візуальні характеристики об'єктів, легко інтегруватися з іншими бібліотеками, такими як NumPy**,** TensorFlow**,** Keras, що розширює можливості побудови повноцінної системи глибинного навчання.

Завдяки таким властивостям OpenCV дозволяє ефективно класифікувати об’єкти, що надходять у форматі зображень або відео, зокрема різні категорії побутових відходів.

Як зазначалося у попередньому розділі, для реалізації системи буде побудована згорткова нейронна мережа**.** Вона виконуватиме класифікацію об'єктів на відео, а бібліотека OpenCV буде використовуватися для захоплення кадрів, їх попередньої обробки та передачі до моделі, а також для візуалізації результатів класифікації в реальному часі.

3.4 Вибір алгоритму оптимізації для навчання згорткової нейронної мережі

Одним із ключових етапів побудови ЗНМ є вибір алгоритму оптимізації, оскільки саме оптимізатор відповідає за оновлення ваг мережі на основі градієнтів функції втрат. Від правильного вибору оптимізатора залежить не лише швидкість збіжності, але й точність та стабільність навчання моделі. Найпоширенішими оптимізаторами, що використовуються у глибокому навчанні:

Наприклад, Stochastic Gradient Descent це базовий алгоритм, який оновлює ваги мережі після обробки кожного прикладу навчальної вибірки, але має повільну збіжність та велику чутливість до налаштувань learning rate.

Також є Momentum розширення SGD, яке враховує попередні напрямки зміни ваг, що дозволяє пришвидшити збіжність і уникати локальних мінімумів.

Далі розглянемо RMSprop. Він використовує експоненційне згладжування квадратів градієнтів для адаптації learning rate, що особливо ефективно при роботі з рекурентними нейронними мережами.

Один із найпоширеніших алгоритмів Adaptive Moment Estimation. Він автоматично підлаштовує швидкість навчання для кожного параметра окремо, що робить його стабільним і ефективним варіантом для багатьох завдань.

Він демонструє стабільність під час навчання, потребує менше налаштувань гіперпараметрів порівняно з іншими алгоритмами та сприяє уникненню проблеми розбіжності градієнтів.

Основна ідея Adam полягає в тому, щоб адаптувати швидкість навчання індивідуально для кожного параметра. Алгоритм використовує експоненційно зважене середнє значення градієнта та його квадрата для оновлення параметрів моделі. Завдяки цьому швидкість навчання автоматично підлаштовується для кожного параметра з урахуванням попередніх значень градієнта.

На рисунку 3.7 наведено графік порівняння ефективності різних оптимізаторів у вигляді функції втрат протягом епох навчання. Як видно, Adam демонструє найменше значення втрат і найстабільніше зниження, що свідчить про високу ефективність алгоритму.

У зв’язку з цим, для вирішення завдання сортування сміття за допомогою комп’ютерного зору, було прийнято рішення використовувати оптимізатор Adam як основний у процесі навчання згорткової нейронної мережі.

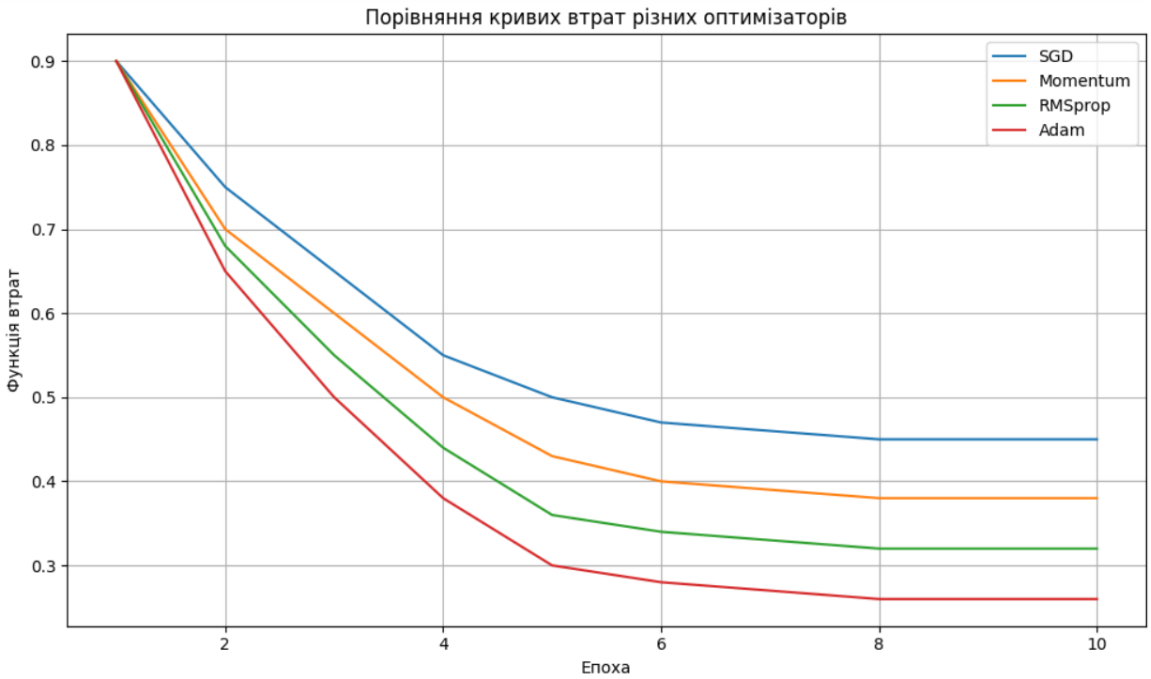


Рисунок 3.7 – Порівняння ефективності оптимізаторів за функцією втрат

Висновки до розділу 3

У цьому розділі був проведений аналіз можливих архітектур нейронних мереж, методів їх навчання, вибору вхідних даних, а також інструментів програмної реалізації, з метою обґрунтування вибору оптимальної моделі для вирішення задачі автоматизованого сортування сміття на основі зображень.

На основі аналізу таких архітектур, як штучні нейронні мережі, рекурентні нейронні мережі і згорткові нейронні мережі , було обґрунтовано доцільність використання CNN як основи моделі. Тому що є найефективнішим типом мереж для задач, пов’язаних із комп’ютерним зором. Завдяки здатності автоматично виділяти ознаки зображення, CNN дозволяють досягати високої точності при класифікації навіть у складних умовах, без необхідності ручного проєктування ознак. Крім того було розглянуто сучасну архітектуру YOLO, яка поєднує в собі переваги згорткових мереж із високою швидкістю обробки.

Ключовим аспектом розробки моделі стало також правильне формування навчального датасету. У ході дослідження було проаналізовано кілька наявних відкритих датасетів і в результаті сформовано об'єднаний датасет обсягом понад 35 000 зображень, що охоплює основні категорії відходів. Такий підхід дозволяє не

лише покращити узагальнюючу здатність моделі, а й зменшити ймовірність перенавчання.

Для реалізації всієї системи було обрано бібліотеку OpenCV, яка забезпечує повний набір інструментів для обробки зображень та відео, інтегрується з фреймворками глибокого навчання, а також дозволяє ефективно працювати з потоковими даними.

Окрему увагу було приділено вибору алгоритму оптимізації, який суттєво впливає на якість та стабільність навчання. Серед розглянутих варіантів — SGD, Momentum, RMSprop і було обґрунтовано вибір оптимізатора Adam, який показав найвищу стабільність, найменше значення функції втрат та найшвидшу збіжність. Adam адаптивно налаштовує learning rate для кожного параметра на основі історії градієнтів, що є критично важливим у навчанні великих згорткових мереж.

4 РЕАЛІЗАЦІЯ НЕЙРОННОЇ МЕРЕЖІ

4.1 Побудова моделі

Як вже було сказано, обрано CNN модель. Побудована модель є послідовною, створеною за допомогою класу Sequential з бібліотеки Keras. Мережа приймає на вхід зображення з трьома каналами — кольорове зображення у форматі RGB. Архітектура складається з чотирьох згорткових, за якими слідує три повнозв’язних шари.

Такий підхід дозволяє моделі поступово виділяти ключові просторові ознаки зображення та перетворювати їх на векторні представлення для ефективної класифікації. Застосування нормалізації, активаційних функцій та регуляризації сприяє стабільному навчанню та підвищенню якості розпізнавання.

Перший шар — це згортковий шар Conv2D, що містить 64 фільтри з розміром ядра 3x3 та параметром padding="same". Цей шар отримує на вхід зображення розміром 48x48x3 і формує карти ознак розміром 48x48x64. Оскільки використовується параметр padding="same", розмір вихідних карт не змінюється порівняно з розміром вхідного зображення.

Шар також використовує активацію ReLU, що дозволяє моделі навчати нелінійні залежності, а також застосовує техніку нормалізації пакетів (BatchNormalization), щоб покращити процес навчання та стабілізувати ваги.

Згортковий шар супроводжується операцією максимального підсумовування (MaxPooling2D) з розміром вікна 2x2, що зменшує розмір вихідних карт ознак до 24x24x64. Також застосовується регуляризація через випадкове вимикання нейронів з ймовірністю 0.25, що допомагає уникнути перенавчання.

Другий згортковий шар також є Conv2D, але з 128 фільтрами та ядром 3x3. Вихідний розмір карт ознак становить 24x24x128. Знову використовується нормалізація пакетів (BatchNormalization), активація ReLU, операція максимального підсумовування (MaxPooling2D з вікном 2x2), що зменшує розмір до 12x12x128. Техніка регуляризації через Dropout (з імовірністю 0.25) застосовується після цього шару.

Третій згортковий шар має 512 фільтрів, ядро 3x3 і параметр padding="same". Цей шар отримує карти ознак розміром 12x12x128 і формує 512 карт ознак, що зберігають той самий розмір — 12x12x512. Після цього шару також застосовуються BatchNormalization, ReLU, MaxPooling2D (зменшує розмір до 6x6x512) та Dropout (0.25).

Четвертий згортковий шар, як і попередній, має 512 фільтрів, ядро 3x3 і параметр padding="same". Цей шар працює з картами ознак розміром 6x6x512 та формує 512 карт ознак розміром 6x6x512. Знову використовується BatchNormalization, ReLU, MaxPooling2D (яке зменшує розмір до 3x3x512) та Dropout (0.25).

Шар Flatten перетворює багатовимірні карти ознак (3x3x512) в одномірний вектор розміром 4608 елементів, що дозволяє передати ці дані до наступного повнозв'язного шару для подальшої класифікації.

Перший повнозв'язний шар (Dense) має 256 нейронів і отримує на вхід вектор розміром 4608 елементів. Цей шар формує компактне представлення зображення, зменшуючи розмір до 256. Після цього застосовується BatchNormalization, активація ReLU та регуляризація через Dropout (0.25).

Другий повнозв'язний шар містить 512 нейронів. Цей шар обробляє дані з попереднього шару і формує більш узагальнене представлення. Після цього використовуються BatchNormalization, ReLU та Dropout (0.25) для стабілізації процесу навчання та запобігання перенавчанню.

Вихідний шар (Dense) складається з 9 нейронів, що відповідають кількості класів для класифікації.

Використовується функція активації Softmax, яка перетворює вихідні значення в ймовірності для кожного класу, дозволяючи моделі вибирати найбільш ймовірний клас.

Архітектура згорткової нейронної мережі, описаної вище, детально представлена в таблиці 4.1.

Таблиця 4.1 − Архітектура моделі згорткової мережі

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 48, 48, 64)	1,792
batch_normalization (BatchNormalization)	(None, 48, 48, 64)	256
activation (Activation)	(None, 48, 48, 64)	0
max_pooling2d (MaxPooling2D)	(None, 24, 24, 64)	0
dropout (Dropout)	(None, 24, 24, 64)	0
conv2d_1 (Conv2D)	(None, 24, 24, 128)	73,856
batch_normalization_1 (BatchNormalization)	(None, 24, 24, 128)	512
activation_1 (Activation)	(None, 24, 24, 128)	0
max_pooling2d_1 (MaxPooling2D)	(None, 12, 12, 128)	0
dropout_1 (Dropout)	(None, 12, 12, 128)	0
conv2d_2 (Conv2D)	(None, 12, 12, 512)	590,336
batch_normalization_2 (BatchNormalization)	(None, 12, 12, 512)	2,048
activation_2 (Activation)	(None, 12, 12, 512)	0
max_pooling2d_2 (MaxPooling2D)	(None, 6, 6, 512)	0
dropout_2 (Dropout)	(None, 6, 6, 512)	0
conv2d_3 (Conv2D)	(None, 6, 6, 512)	2,359,808
batch_normalization_3 (BatchNormalization)	(None, 6, 6, 512)	2,048
activation_3 (Activation)	(None, 6, 6, 512)	0
max_pooling2d_3 (MaxPooling2D)	(None, 3, 3, 512)	0
dropout_3 (Dropout)	(None, 3, 3, 512)	0
flatten (Flatten)	(None, 4608)	0
dense (Dense)	(None, 256)	1,179,904
batch_normalization_4 (BatchNormalization)	(None, 256)	1,024
activation_4 (Activation)	(None, 256)	0
dropout_4 (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 512)	131,584
batch_normalization_5 (BatchNormalization)	(None, 512)	2,048
activation_5 (Activation)	(None, 512)	0

Layer (type)	Output Shape	Param #
dropout_5 (Dropout)	(None, 512)	0
dense_2 (Dense)	(None, 9)	4,617

Загальна кількість параметрів у моделі становить понад 4 млн (4,349,833), що свідчить про її високу ємність та потенціал для моделювання складних патернів у вхідних даних. З них більша частина — 4,345,865 параметрів — є тренованими, тобто підлягають оптимізації під час навчання моделі, тоді як 3,968 параметрів залишаються нетренованими, що пов’язано з використанням шарів нормалізації пакетів.

4.2 Тренування моделі глибокого навчання

Для сортування сміття за допомогою комп’ютерного зору було навчено згорткову нейронну мережу.

Для покращення здатності моделі до узагальнення, було використано техніку аугментації зображень. За допомогою класу ImageDataGenerator з бібліотеки Keras були реалізовані такі трансформації: горизонтальне та вертикальне віддзеркалення, обертання зображень, масштабування, зсуви по ширині та висоті, зміщення по зсувній сітці.

Цей підхід дозволяє застосовувати різноманітні трансформації до зображень у реальному часі, не зберігаючи нові копії файлів, що значно економить пам’ять і підвищує ефективність процесу навчання.

При навчанні моделей машинного навчання важливо не лише тренувати модель на наявних даних, а й перевіряти її здатність до узагальнення на нових, раніше не бачених прикладах. Це дозволяє уникнути перенавчання (overfitting). Для цього набір даних зазвичай розбивається на тренувальний (training) та валідаційний (validation) піднабори.

Для автоматичного розподілу даних між цими піднабором використовувався параметр validation_split, що належить класу ImageDataGenerator. Всі пікселі зображень були нормалізовані до діапазону [0, 1].

Тренувальні дані складають 80%, а валідаційні — 20%. Отже, 20% усіх зображень у кожній категорії використовуються для валідації, а решта 80% — для тренування. Завдяки цьому не потрібно вручну створювати окремі папки для валідаційних даних, що спрощує організацію структури проєкту. Параметр target_size вказує розмір вхідного зображення.

Також використовуються такі параметри як horizontal_flip=True — активує горизонтальне віддзеркалення зображень. Кожне зображення має шанс бути перевернутим ліворуч/праворуч.

А vertical_flip=True — активує вертикальне віддзеркалення. rotation_range = 15 — задає межу випадкового обертання зображень в межах ±15 градусів. zoom_range = 0.1 — дозволяє моделі краще адаптуватися до змін розміру об'єкта. width_shift_range = 0.15 — дозволяє зсувати зображення по ширині на 15%. height_shift_range = 0.15 — дозволяє зсувати зображення по висоті на 15%. І shear_range = 0.1 виконує перетворення, схоже на нахил або деформацію зображення по діагоналі. fill_mode='nearest' — визначає, як заповнювати порожні пікселі після трансформацій. rescale=1./255 — нормалізує пікселі зображення.

Завдяки цим параметрами генератори даних готові до використання для навчання та перевірки моделі. На рисунку 4.1 зображено метод генерації даних.

```
train_datagen = ImageDataGenerator(rescale=1./255)
val_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(48, 48),
    batch_size=128,
    class_mode='categorical',
    color_mode='rgb',
    shuffle=True
)

val_generator = val_datagen.flow_from_directory(
    val_dir,
    target_size=(48, 48),
    batch_size=128,
    class_mode='categorical',
    color_mode='rgb',
    shuffle=False
)
```

Рисунок 4.1 − Метод генерації даних

Модель була скомпільована за допомогою функції втрат categorical_crossentropy та оптимізатора Adam. Для оцінки якості навчання була обрана метрика точності (accuracy). Для уникнення перенавчання використовувались колбеки: ModelCheckpoint, EarlyStopping і ReduceLROnPlateau.

Існують функції, які допомагають контролювати процес навчання, зупиняти його, змінювати параметри або зберігати модель, їх називають колбеками. Колбек ModelCheckpoint автоматично зберігає найкращу версію моделі під час навчання. Найкраща модель — це та, яка досягла найвищої точності на валідаційній вибірці. Параметр save_best_only=True гарантує, що файл моделі не буде переписано гіршими версіями, а параметр mode='max' означає, що найкращою вважається модель з максимальною метрикою точності.

Колбек EarlyStopping зупиняє навчання, якщо на валідаційних даних не спостерігається покращення протягом заданої кількості епох. Додатково були використані параметри: monitor='val_loss' для спостереження за валідаційною втратою, patience=3 для визначення, скільки епох модель може погіршуватися, перш ніж зупинити навчання, і restore_best_weights=True, щоб після зупинки модель автоматично повернулася до найкращих ваг.

Колбек ReduceLROnPlateau дозволяє зменшити швидкість навчання, якщо валідаційна помилка не покращується. Для цього були використані параметри: monitor='val_loss' використовується для спостереження за валідаційною помилкою, а factor=0.2 використовується для зменшення швидкості навчання в 5 разів, patience=5, щоб зменшення відбулося після 5 епох без покращення, та min_lr=1e-6 для встановлення мінімальної швидкості навчання. На рисунку 4.2 зображено використання callback-функцій для оптимізації навчання.

```
callbacks = [
    keras.callbacks.ModelCheckpoint('GarbageModel.keras', save_best_only=True, monitor='val_accuracy'),
    keras.callbacks.ReduceLROnPlateau(monitor='val_accuracy', factor=0.2, patience=3, verbose=1),
    keras.callbacks.EarlyStopping(monitor='val_accuracy', patience=5, restore_best_weights=True, verbose=1)
]
```

Рисунок 4.2 – Використання callback-функцій для оптимізації навчання

Далі необхідно скомпілювати модель це означає, що неохідно визначити, за якою функцією втрат вона буде оптимізуватись, який оптимізатор буде використовуватись для оновлення ваг, та яку метрику використовувати для оцінки якості навчання.

Оскільки задача класифікації є багатокласовою, було використано функцію втрат Categorical Crossentropy (категоріальна кросентропія).

Як вже було написано необхідно використовувати оптимізатор Adam, який автоматично коригує швидкість навчання на основі першого та другого моментів градієнтів.

Для оцінки якості моделі під час навчання та валідації використовується метрика accuracy. Вона показує, яка частка передбачень моделі була правильними. У випадку багатокласової класифікації, точність обчислюється як відсоток правильно класифікованих зображень у порівнянні з усіма прикладами. На рисунку 4.3 зображено компіляція моделі.

```
optimizer = keras.optimizers.Adam(learning_rate=1e-4)
model.compile(optimizer=optimizer, loss='categorical_crossentropy', metrics=['accuracy'])
```

Рисунок 4.3 − Компіляція моделі

Далі відбувається ключовий етап — навчання моделі. Навчання здійснюється за допомогою методу fit(), який відповідає за організацію циклу тренування. Цей цикл повторюється протягом заздалегідь визначеної кількості епох — повних проходів по всьому тренувальному набору даних.

Під час кожної епохи модель виконує передбачення на тренувальних зразках, порівнює їх з правильними відповідями, обчислює функцію втрат і оновлює ваги за допомогою обраного оптимізатора. Після завершення проходу по всьому тренувальному набору, модель оцінюється на валідаційному наборі, який не використовувався для навчання. Це дозволяє контролювати якість узагальнення та вчасно виявляти перенавчання (overfitting).

За допомогою методу fit() також можна вказати розмір батча — кількість зразків, які модель обробляє одночасно перед оновленням ваг. Оптимальний вибір батча впливає на швидкість та стабільність навчання.

Навчання моделі — це процес, у якому модель покращує свої параметри на основі зворотного зв'язку, отриманого від функції втрат, і перевіряє свої результати на валідаційному наборі, що забезпечує ефективне навчання та підвищення якості передбачень. На рисунку 4.4 зображено навчання моєї моделі.

```
history = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=60,
    callbacks=callbacks
)
```

Рисунок 4.4 – Навчання моделі

Схему алгоритму навчання мережі можливо побачити на кресленику ІК13.100БАК.006 Д1. На ній зображений повний процес підготовки та навчання моделі згорткової нейронної мережі.

Процес починається з ініціалізації середовища розробки та підготовки необхідних ресурсів. Першим кроком є завантаження зображень, які розподілені за класами, це зроблено для чіткого співвідношення кожного зображення до певної категорії.

Далі здійснюється попередня обробка даних, яка включає зміну розміру зображень до стандартного формату, нормалізацію піксельних значень, а також, за потреби, розширення даних для підвищення їх різноманітності.

Після попередньої обробки дані розподіляються на навчальну та валідаційну вибірки, це робиться для того, щоб контролювати процес навчання та запобігати перенавчанню моделі.

На основі підготовлених вибірок створюються генератори даних, що формують батчі у реальному часі, оптимізуючи використання пам'яті та обчислювальних ресурсів.

Далі побудова архітектури нейронної мережі. Формується структура моделі, яка містить шари згортки, пулінгу, активації, нормалізації та повнозв'язні шари для виконання класифікації на виході.

Після завершення етапу підготовки розпочинається процес навчання моделі. Перед стартом тренування налаштовуються функції зворотного виклику (callbacks), які включають збереження найкращої версії моделі, ранню зупинку навчання при погіршенні результатів та адаптивне регулювання швидкості навчання.

Процес тренування моделі відбувається за допомогою навчальної вибірки. Після кожної епохи модель проходить оцінку на валідаційній вибірці для контролю за її якістю.

За результатами оцінки проводиться перевірка досягнутої точності:

якщо точність є задовільною, навчання завершується;

якщо ні — модель продовжує навчання з оновленими параметрами або із застосуванням змінених стратегій оптимізації.

Кінцевим етапом є завершення процесу навчання після досягнення бажаного рівня точності або вичерпання заданої кількості епох.

Висновки до розділу 4

На основі аналізу та опису реалізації ЗНМ для задачі сортування сміття, було побудовано та навчено ефективну модель, яка здатна розпізнавати 9 класів об’єктів. Було створено послідовну CNN модель із використанням бібліотеки Keras, що складається з чотирьох згорткових шарів із застосуванням BatchNormalization, активації ReLU, MaxPooling та Dropout для підвищення стабільності навчання і запобігання перенавчанню.

Після згорткових шарів модель містить три повнозв’язані (Dense) шари, які формують векторне подання ознак і реалізують класифікацію за допомогою

softmax активації. Архітектура моделі детально спроєктована для поступового вилучення важливих ознак із вхідних RGB зображень розміром 48x48.

Для підвищення здатності моделі до узагальнення було використано аугментацію зображень із різними трансформаціями (віддзеркалення, обертання, масштабування, зсуви), реалізовану через ImageDataGenerator, що дозволяє динамічно збільшувати різноманітність тренувальних даних без додаткового використання пам’яті.

Навчання моделі відбувалося із застосуванням функції втрат categorical_crossentropy, оптимізатора Adam та метрики точності (accuracy). Для контролю навчання були використані callback-функції ModelCheckpoint, EarlyStopping і ReduceLROnPlateau, що забезпечують автоматичне збереження найкращої версії моделі, ранню зупинку при відсутності покращення та адаптивне зменшення швидкості навчання.

Реалізована модель має високу ємність понад 4 млн параметрів, що дозволяє їй вивчати складні просторові залежності у вхідних даних. Використання аугментації та регуляризації допомагає уникнути перенавчання та підвищити узагальнювальну здатність моделі. Стабільність та ефективність навчання підтримуються нормалізацією пакетів і адаптивним управлінням швидкістю навчання.

Завдяки описаному підходу побудови, підготовки та навчання моделі досягається висока точність класифікації, що підтверджується результатами валідації.

Така модель може бути застосована для автоматизації сортування сміття, що є важливим кроком у напрямку екологічної безпеки та ефективного використання ресурсів.

5 ПРОЄКТУВАННЯ СИСТЕМИ ДЛЯ СОРТУВАННЯ СМІТТЯ

5.1 Вибір засобів для розробки

Для роботизованої системи сортування сміття було прийнято рішення обрати мову програмування Python, оскільки він є популярною мовою програмування для різних задач машинного навчання і комп'ютерного бачення, тому що має велику кількість різних бібліотек.

Крім того ця мова має доволі простий синтаксис. І завдяки Python можливо ефективно реалізувати завдання збору даних, обробки зображень, побудови моделі та її навчання.

Для створення, тренування та оцінки нейронної мережі було використано бібліотеки TensorFlow та її бібліотку Keras.

Зручна платформа з відкритим кодом для машинного навчання що використовує графи потоків даних це TensorFlow. У такому графі вузли представляють математичні операції, а ребра — багатовимірні масиви даних (тензори), які передаються між цими операціями.

Ця гнучка архітектура дозволяє описувати алгоритми машинного навчання як граф пов'язаних операцій. Їх можна навчати та виконувати на графічних процесорах, центральних процесорах та процесорах на різних платформах без переписування коду, від портативних пристроїв до настільних комп'ютерів та високопродуктивних серверів [16].

Високорівневий API глибокого навчання, який спрощує процес побудови глибоких нейронних мереж це Keras. Спочатку розроблений як незалежна бібліотека, Keras тепер тісно інтегрований у TensorFlow як його офіційний високорівневий API. Він підтримує кілька серверних механізмів, включаючи TensorFlow, Theano та CNTK. Keras спрощує визначення, навчання та оцінку моделей глибокого навчання без необхідності глибоких знань низькорівневих операцій [17].

Ці інструменти забезпечують потужні можливості для побудови згорткових нейронних мереж CNN, зручні засоби для налаштування архітектури моделі та оптимізації процесу навчання.

Бібліотека OpenCV використовується для роботи із зображеннями у реальному часі, тобто для захоплення кадрів з вебкамери та їх попередньої обробки.

Вона дозволяє швидко отримувати зображення, виконувати базову обробку та підготовку даних до класифікації.

Багатоплатформна подійно-орієнтована бібліотека Python за допомогою якої можна створити графічний користувацький інтерфейс це Tkinter. Він надає прості для використання інструменти, щоб створювати унікальні та зручні інтерфеси та елементи інтерфейсу, такі як кнопки, мітки меню та інше.

Зручна платформа для проєктів з машинного навчання – Kaggle. Вона дає доступ до багатьох і різноматніх датасетів, також можна самому завантажувати різні набори даних. Етапи розробки і тренування моделі проводились у цьому середовищі.

Це забезпечило гнучкість у експериментах, можливість швидкої перевірки гіпотез та використання апаратних ресурсів хмарних платформ таких як GPU пришвидшує обчислення в задачах глибокого навчання і TPU ще швидший, ніж GPU.

Для ефективної роботи з масивами даних та здійснення чисельних обчислень використовуємо бібліотеку NumPy. Вона дозволила оптимізувати обробку великих обсягів зображень і прискорити виконання операцій над матрицями.

Для візуалізації результатів навчання моделі, графіки точності та функції втрат на тренувальних і валідаційних даних було застосовано бібліотеку Matplotlib.

Як середовище розробки було обрано Visual Studio Code, оскільки його зручно використовувати як для розробки, так і для впровадження навченої машинної моделі.

Крім того, Visual Studio Code зручно використовувати в контексті машинного навчання, оскільки він підтримує велику кількість розширень, зокрема для роботи з TensorFlow, PyTorch, Keras та іншими бібліотеками, що значно спрощує процес розробки ML-проєктів.

5.2 Реалізація роботизованої системи сортування сміття

Роботизована система сортування сміття побудована на нейронній мережі та класифікація здійснюється в режимі реального часу на основі візуального аналізу зображень або відеопотоку. Ця система дозволяє користувачі класифікувати сміття з фотографій, відео або з веб-камери. На кресленику ІК13.100БАК.006 Д2 представлена схема алгоритму роботи системи.

Спочатку завантажується попередньо натренована модель нейронної мережі з розширенням .keras, яка класифікує об’єкти сміття. Далі відбувається відображення графічного інтерфейсу користувача GUI, створеного за допомогою бібліотеки Tkinter. Користувач має змогу обрати джерело вхідних даних: статичне зображення, відеофайл або живий потік із камери.

Функція classify_video(source) для класифікації з відео або камери. Вона відкриває відеопотік і для кожного кадру викликає classify_frame() для класифікації та накладає текстову мітку на зображення.

Далі функція classify_image() для класифікації з зображення. Вона відкриває діалог вибору зображення, зчитує його через cv2.imred() та передає кадр у функцію classify_frame() для розпізнавання і потім виводиться також результат у вікні.

Функція: classify_frame(frame) є універсальним ядром системи розпізнавання, яке використовується як для фото, так і для відео. Її основне призначення — підготувати вхідні дані та отримати передбачення від моделі нейронної мережі. Вона масштабує зображення до 224x224 та передає кадр у модель для передбачення і повертає мітку класу з імовірністю. І є універсальною, тому що один код працює і для відео, і для зображень.

Тобто функція classify_frame(frame) — це точка входу до моделі машинного навчання. Яка виконує передбачення, а classify_video(source) і classify_image() — це інтерфейсні функції, які відповідають за отримання зображень - вхідних даних з різних джерел: відео, камери або файлу. Також створено графічний інтерфейс за допомогою Tkinter з кнопками завантажити відео, увімкнути камеру, завантажити зображення.

5.2.1 Опис і тестування програмного додатку

Цей проєкт є повноцінним програмним додатком, оскільки має графічний інтерфейс користувача, працює як самостійний застосунок та має чітку архітектуру. На рисунку 5.1 зображено інтерфес користувача.

При відкритті цього програмного додатку користувач бачить графічний інтерфейс, який є доволі простим але інтуїтивно зрозумілим.

Основне завдання це надати користувачеві зручний доступ до функціоналу системи класифікації сміття.

Інтерфейс містить вікно програми з нейтральним фоном світло-сірого кольору та всі елементи керування знаходяться у центрі вікна.

Кожна дія: відео, камера чи зображення виконується у паралельному потоці Thread для уникнення зависання графічного інтерфейсу під час обробки.

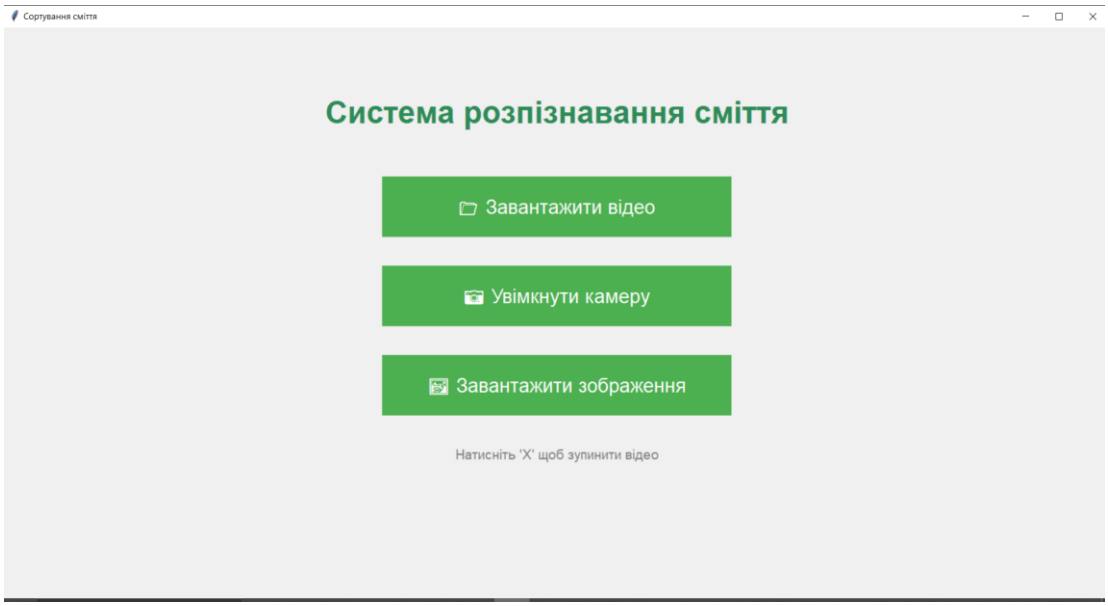


Рисунок 5.1 – Інтерфейс користувача

Користувачу доступні три головні кнопки. Після вибору однієї з цих функцій запускається класифікація об’єктів і відображення мітки класу сміття, рівень впевненості і виведення інформації у вигляді підпису зверху зображення або відео.

Для того, щоб протестувати одну з цих функцій потрібно двічі натиснути на кнопку. Під кнопками розміщена підказка для користувача: «Натисніть 'X' щоб зупинити відео».

Наприклад кнопка «Завантажити відео», при її натисканні відкривається діалогове вікно для вибору відеофайлу і після вибору файлу, то відео зчитується покадрово з потоку з камери і виконується класифікація кожного кадру і далі виводиться результат на екрані, як можемо побачити це на рисунку 5.2



Рисунок 5.1 – Відображення сортування сміття з відеофайлу

Для завершення перегляду користувачу потрібно натиснути клавішу 'X', щоб зупинити класифікацію і закрити вікно OpenCV.

Далі протестуємо кнопку «увімкнути камеру», при її натисканні активується вебкамера пристрою та відкривається діалогове вікно для вибору відеофайлу. Відеофайли підтримуються у форматах .mp4, .avi.

В реальному часі захоплюються кадри, які класифікуються, а результати накладаються поверх зображення, як можемо побачити на рисунку 5.2

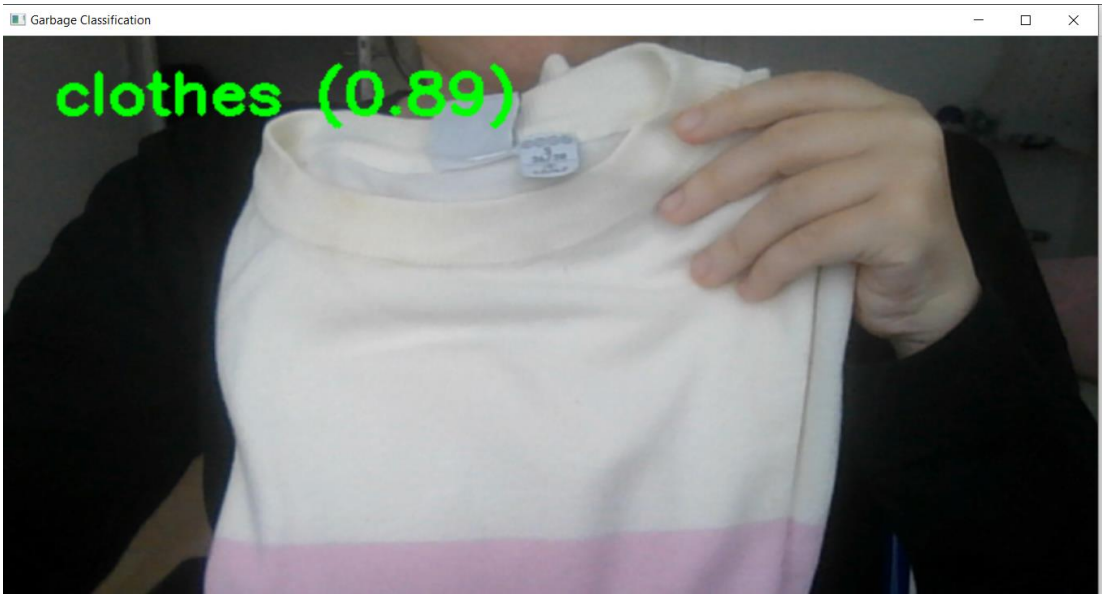


Рисунок 5.2 – Відображення сортування сміття в режимі реального часу

Далі протестуємо кнопку «Завантажити зображення». Ця функція реалізує можливість класифікації сміття на основі статичних зображень, і не потрібно підключати відео або камеру.

При натисканні цієї кнопки відкривається типове діалогове вікно вибору файлу, де необхідно обрати зображення формату .jpg, .png, .jpeg. та потім здійснюється класифікація. Зображення відображається у вікні з підписом результату класифікації, як бачимо це на рисунку 5.3

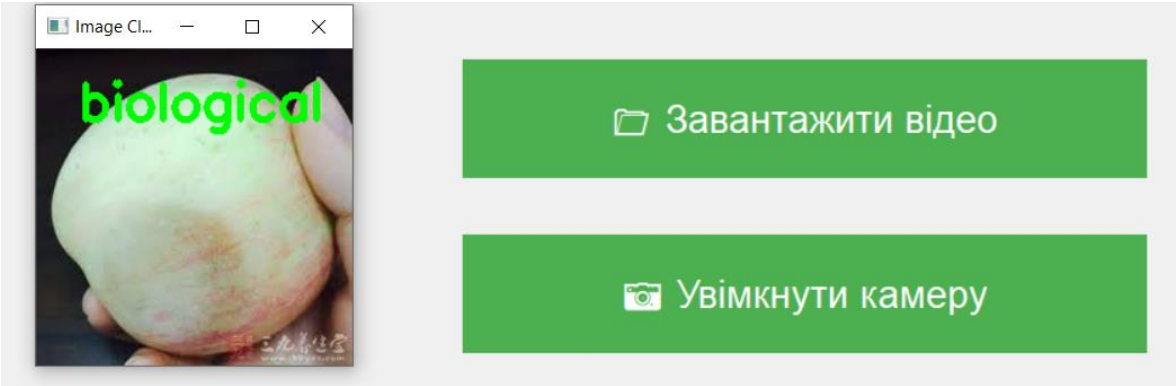


Рисунок 5.3 – Відображення сортування сміття з зображення

Діаграма послідовностей представлена на кресленику ІК13.100БАК.006 Д4, яка ілюструє процес взаємодії між основними компонентами системи сортування

сміття, реалізованої з використанням графічного інтерфейсу Tkinter, бібліотеки комп'ютерного зору OpenCV та згорткової нейронної мережі.

Взаємодія починається з користувача, який ініціює процес, натискаючи одну з кнопок інтерфейсу — для завантаження відео, зображення або активації камери. Графічний інтерфейс (GUI), створений за допомогою Tkinter, забезпечує зручний засіб взаємодії з системою та запуску процесу класифікації.

Обробка зображень та взаємодія з бібліотекою OpenCV і моделлю нейронної мережі здійснюється через модуль класифікатора. Саме він відповідає за запуск класифікації у відповідному потоці, отримує кадри чи зображення, готує їх до аналізу та координує наступні дії. Завдяки мультипоточній обробці забезпечується швидкість і стабільність роботи системи навіть при обробці відеопотоку в реальному часі.

Бібліотека OpenCV виконує попередню обробку зображення: масштабування до стандартного розміру, нормалізацію та нанесення текстової інформації. Після підготовки зображення передається до попередньо навченої згорткової нейронної мережі CNN, яка виконує класифікацію сміття за відповідними категоріями. Результатом є мітка класу та рівень впевненості у визначенні.

Оброблене зображення з результатами класифікації повертається до інтерфейсу GUI, де виводиться у вікні програми для ознайомлення користувача. Такий підхід забезпечує інтерактивність і зворотний зв'язок, що є важливою складовою ефективного функціонування системи. У результаті взаємодії всіх компонентів досягається мета — автоматизоване сортування сміття з мінімальним втручанням людини.

Діаграма варіантів використання представлена на кресленику ІК13.100БАК.006 Д3 відображає функціональні можливості інтелектуальної системи сортування сміття, яка працює на основі методів штучного інтелекту. Вона охоплює два основні типи користувачів — звичайного користувача та адміністратора системи. Кожен із них має власний рівень доступу та унікальний набір функцій, що дозволяє забезпечити ефективну й гнучку взаємодію з системою.

Звичайний користувач — це кінцевий споживач, чия взаємодія з системою зосереджена на класифікації відходів. Йому надається інтуїтивно зрозумілий інтерфейс, який дозволяє або активувати камеру пристрою для отримання зображення об’єкта в реальному часі, або завантажити відео чи фото з галереї, якщо користувач бажає класифікувати зображення зі свого архіву. Такий підхід робить процес максимально зручним і доступним у повсякденному використанні.

Адміністратор, своєю чергою, має розширені права доступу і відповідає за налаштування, підтримку та розвиток інтелектуальної частини системи. До його обов’язків входить управління наборами класів сміття — їх додавання, редагування або видалення відповідно до потреб користувачів чи нових стандартів. Він також може ініціювати процес навчання моделі на нових даних, оновлювати вже існуючу модель з урахуванням отриманих результатів, а також додавати нові навчальні або тестові вибірки зображень. Крім того, адміністратор слідкує за актуальністю всіх компонентів системи, забезпечуючи її стабільну роботу, та проводить тестування моделі з метою перевірки її точності та ефективності.

Діаграма варіантів використання демонструє повну логіку роботи системи, гармонійно поєднуючи простоту для кінцевих користувачів із широкими можливостями адміністрування. Така архітектура дає можливість не лише результативно вирішувати завдання сортування сміття, а й забезпечує масштабованість, адаптивність та універсальність рішення для різних сфер застосування — від побутових умов до промислових та муніципальних потреб.

5.2.2 Архітектурна здатність до розгортання на мобільному роботі

Розроблена програмна система для класифікації сміття із використанням комп’ютерного зору має універсальну архітектуру, що дозволяє легко адаптувати її до різних типів апаратного забезпечення, зокрема мобільних роботизованих платформ. Завдяки використанню бібліотеки OpenCV для обробки зображень та відео, а також гнучкості Python-середовища, система може бути запущена не лише на персональному комп’ютері, але й на вбудованих пристроях, таких як Raspberry Pi, NVIDIA Jetson, Orange Pi тощо.

Ключова обчислювальна логіка класифікації зображень реалізована у вигляді окремих функцій, які не залежать від графічного інтерфейсу. Це дозволяє легко адаптувати систему для запуску на інших пристроях без інтерфейсу користувача, наприклад, на мобільних роботах.

Взаємодія з користувачем реалізована окремо — через графічний інтерфейс на базі Tkinter, що може бути вилучений або замінений залежно від обраного середовища розгортання.

Система підтримує обробку відеопотоку з будь-якого джерела, зокрема USB-камери або IP-камери, що дозволяє використовувати модуль розпізнавання з камери, закріпленої на мобільному роботі.

При цьому для інтеграції системи в мобільну платформу необхідно лише мінімально адаптувати модель нейронної мережі до обмежень обчислювальних ресурсів, зокрема шляхом її оптимізації, наприклад, перетворення у формат TensorFlow Lite або використання полегшених моделей, таких як MobileNet.

Розроблена система є архітектурно придатною до розгортання на мобільному роботі та може бути основою для створення автономної роботизованої платформи для сортування сміття в умовах реального середовища, де відсутня фіксована інфраструктура.

5.3 Результати роботи роботизованої системи сортування сміття

Для навчання моделі необхідно було натренерувати мережу. В ході навчання було витрачено багато часу і були досліджені різні варіанти гіперпараметрів та побудували графіки. Для оцінки ефективності побудованої моделі сортування сміття було проведено візуалізацію метрик навчання, побудовано матрицю плутанини та тренування мережі на 80 епохах, але завдяки методу ReduceLROnPlateau, навчання зупинилося на 53 епохах, тому що значення функції втрат на валідаційній вибірці не покращилося упродовж 3 епох, можемо це побачити на рисунку 5.4

```
Epoch 48/80
203/203 ━━━━━━━━━━━━━━━━━━━━ 82s 402ms/step - accuracy: 0.8968 - loss: 0.3063 - val_accuracy: 0.8115 - val_loss: 0.6707 - learning_rate: 2.0000e-05
Epoch 49/80
203/203 ━━━━━━━━━━━━━━━━━━━━ 83s 408ms/step - accuracy: 0.8971 - loss: 0.3029 - val_accuracy: 0.8087 - val_loss: 0.6759 - learning_rate: 2.0000e-05
Epoch 50/80
203/203 ━━━━━━━━━━━━━━━━━━━━ 80s 394ms/step - accuracy: 0.9019 - loss: 0.2901 - val_accuracy: 0.8055 - val_loss: 0.6850 - learning_rate: 2.0000e-05
Epoch 51/80
203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 316ms/step - accuracy: 0.9002 - loss: 0.2990
Epoch 51: ReduceLROnPlateau reducing learning rate to 3.999999898951501e-06.
203/203 ━━━━━━━━━━━━━━━━━━━━ 80s 394ms/step - accuracy: 0.9002 - loss: 0.2990 - val_accuracy: 0.8100 - val_loss: 0.6957 - learning_rate: 2.0000e-05
Epoch 52/80
203/203 ━━━━━━━━━━━━━━━━━━━━ 82s 404ms/step - accuracy: 0.9035 - loss: 0.2820 - val_accuracy: 0.8060 - val_loss: 0.6835 - learning_rate: 4.0000e-06
Epoch 53/80
203/203 ━━━━━━━━━━━━━━━━━━━━ 82s 404ms/step - accuracy: 0.9049 - loss: 0.2869 - val_accuracy: 0.8084 - val_loss: 0.6846 - learning_rate: 4.0000e-06
Epoch 53: early stopping
```

Рисунок 5.4 – Результат навчання моделі

На рисунках 5.5 бачимо графік втрат і на рисунку 5.6 графік точності згідно тренувальної та валідаційної вибірок упродовж 53 епох навчання. На графіку точності значення зростають, а на графіку втрат навпаки спадають.

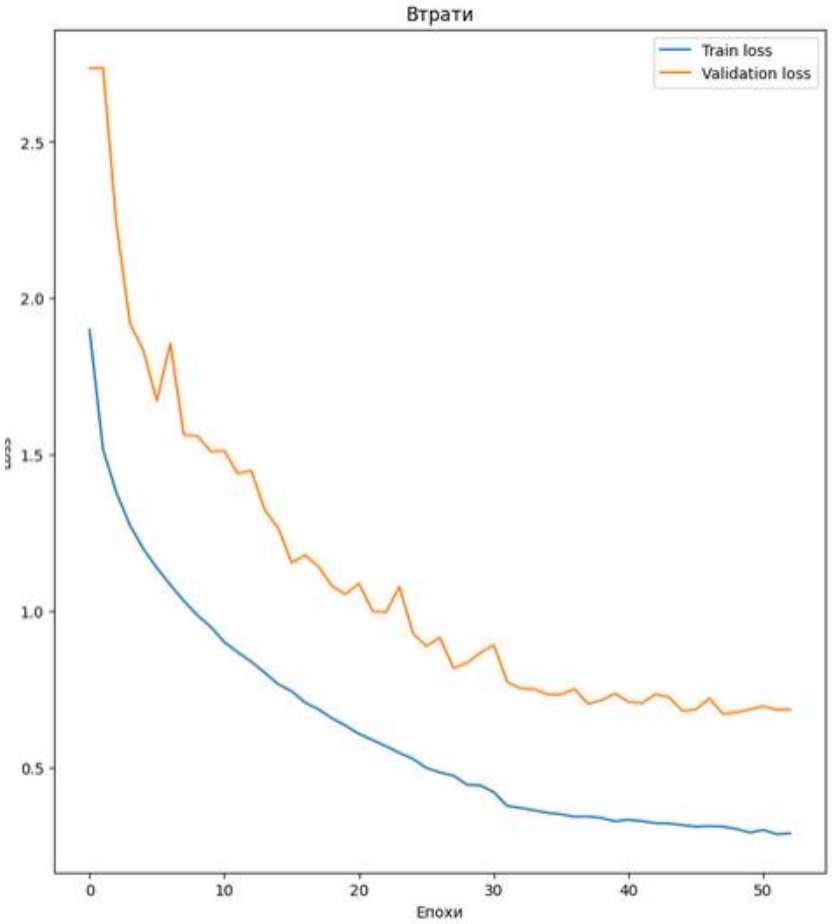


Рисунок 5.5 – Графік втрат

З огляду на графік втрат, можемо побачити що синя лінія, тренувальні дані, постійно зменшується, що свідчить про те, що модель поступово покращує свої передбачення на навчальних даних. А помаранчева лінія, валідаційні дані, також

має загальну тенденцію до зниження, хоча з невеликими коливаннями, крім того validation loss не зростає наприкінці, що є позитивною ознакою.

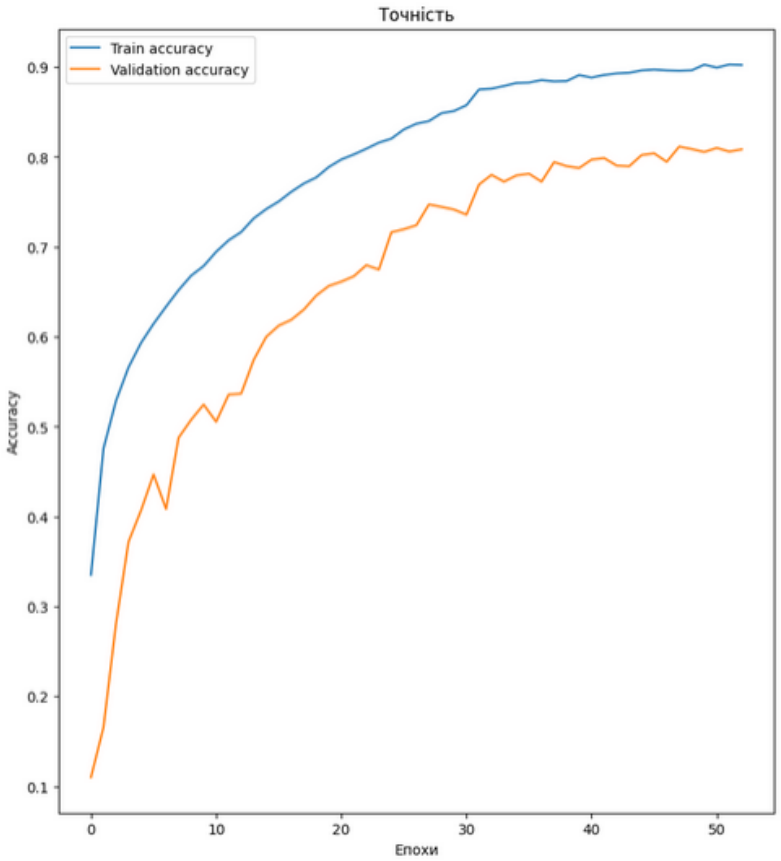


Рисунок 5.6 – Графік точності

Якщо дивитися на графік точності, то видно, що значення на тренувальній вибірці стабільно зростає і наближається до 85% на останніх епохах, а на валідаційній вибірці також зростає, хоча і повільніше, досягаючи близько 78%.

Різниця між train і validation accuracy наприкінці тренування становить приблизно 7%, що є прийнятним для багатокласової задачі класифікації. Модель має хорошу здатність до узагальнення, хоча ще є простір для поліпшення, наприклад, за рахунок додаткової обробки даних.

Отже, графіки втрат та точності демонструють поступове покращення моделі протягом навчання. Тренувальна точність досягає 85%, а валідаційна — близько 78%, що свідчить про добру здатність моделі узагальнювати.

Відсутність різкого зростання валідаційних втрат свідчить про відсутність явного перенавчання, а плавне покращення метрик підтверджує ефективність обраної архітектури та підходу до навчання.

Для кращої оцінки ефективності класифікаційної моделі було побудовано матрицю невідповідностей, зображену на рисунку 5.7. Вона дозволяє наочно оцінити, як часто об'єкти певного класу правильно класифікуються, а також в які інші класи найчастіше помилково відносяться.

Наприклад, модель успішно класифікує об'єкти класу clothes (85% правильних відповідей), а також biological та cardboard (71% та 78% відповідно). А класи glass (54% правильних відповідей), paper (55% правильних відповідей), plastic(55% правильних відповідей), shoes (60% правильних відповідей) Проте є проблеми з класифікацією "metal" – лише 16% вірно класифікованих об’єктів і battery — лише 33% об'єктів були правильно класифіковані, інші часто потрапляли до класів metal, glass або shoes.

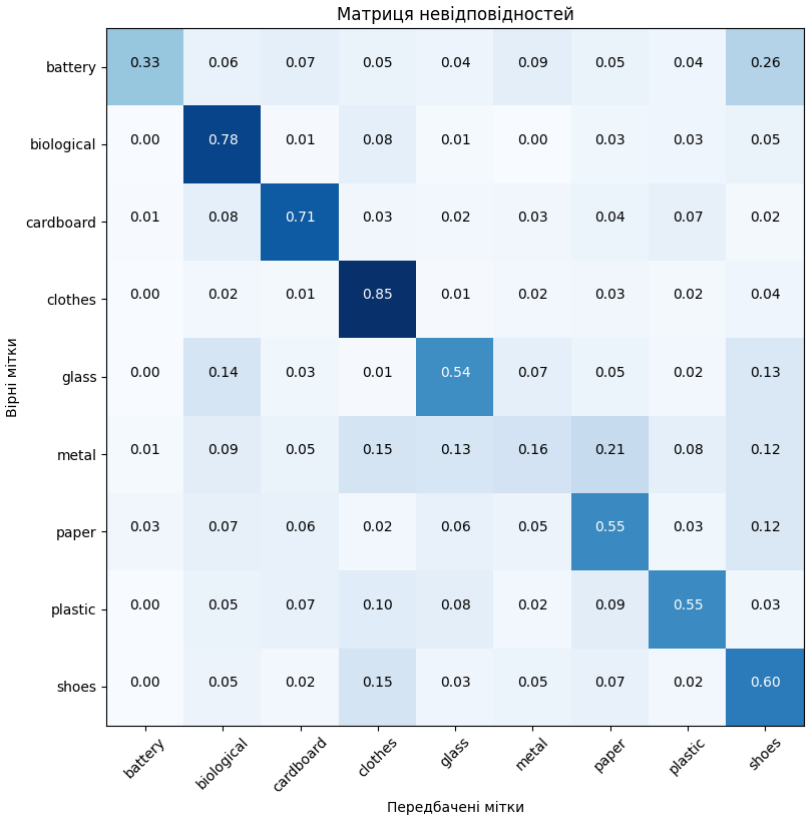


Рисунок 5.7 – Матриця невідповідностей

Загальна точність класифікаційної моделі склала 78%. Отриманий результат свідчить про задовільну, хоча й не ідеальну якість класифікації. Це може бути зумовлено кількома чинниками, серед яких — обмежений обсяг навчального набору, висока схожість візуальних характеристик між окремими класами об’єктів, а також часткове перекриття ознак в ознаковому просторі. Такі фактори можуть ускладнювати точне розмежування між категоріями сміття, особливо в умовах варіативного освітлення, фону чи якості зображень.

Якість класифікації може бути суттєво покращена шляхом збільшення даних датасету кількістю та якісними зображенням для кожного класу, або можна розглянути використання попередньо навченої моделі, яка вже була натренована на великому обсязі даних, наприклад це можуть бути такі моделі: MobileNetV2, ResNet50 або EfficientNet. Це дозволить отримати кращі результати.

Щоб оцінити, наскільки ефективно мережа працює на практиці, потрібно провести її реальне тестування. Створена система сортування сміття має класифікувати такий мусор як бата-рейки, біологічний, коробки, одяг, стекло, метал, папір, пластик, взуття у режимі реального часу з відео, з веб-камери та зображень. Результати роботи роботизованої системи сортування сміття з зображень продемонстровано на рисунку 5.8



Рисунок 5.8 – Демонстрація конкретного розпізнавання сміття з зображень

Для демонстрації можливостей роботизованої системи сортування сміття у режимі реального часу, а також при обробці відео, було використано власні побутові предмети як тестові об’єкти.

Візуальні результати роботи системи під час сортування сміття за зображеннями наведено на рисунку 5.9, а з відео на рисунку 5.10. Це дозволяє оцінити точність і надійність її функціонування в умовах, наближених до реального середовища.

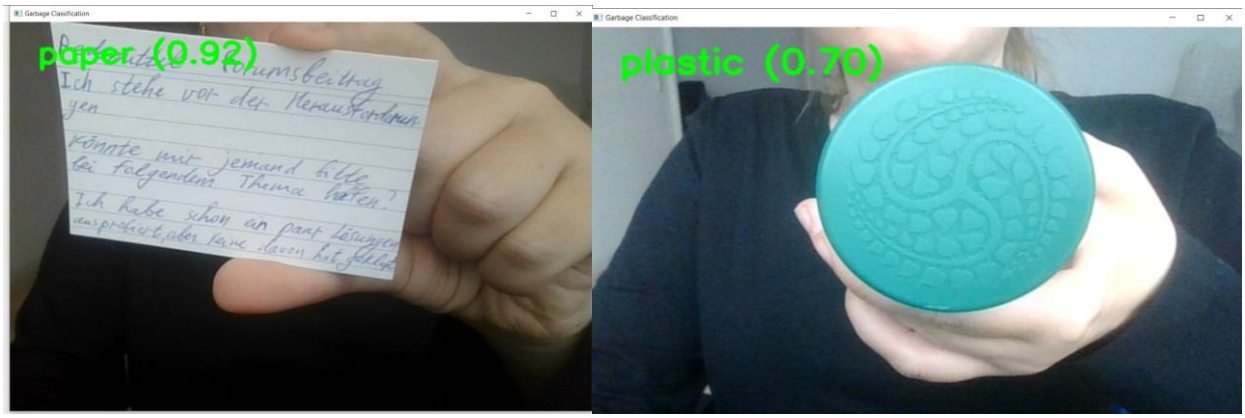


Рисунок 5.9 – Демонстрація розпізнавання сміття в режимі реального часу

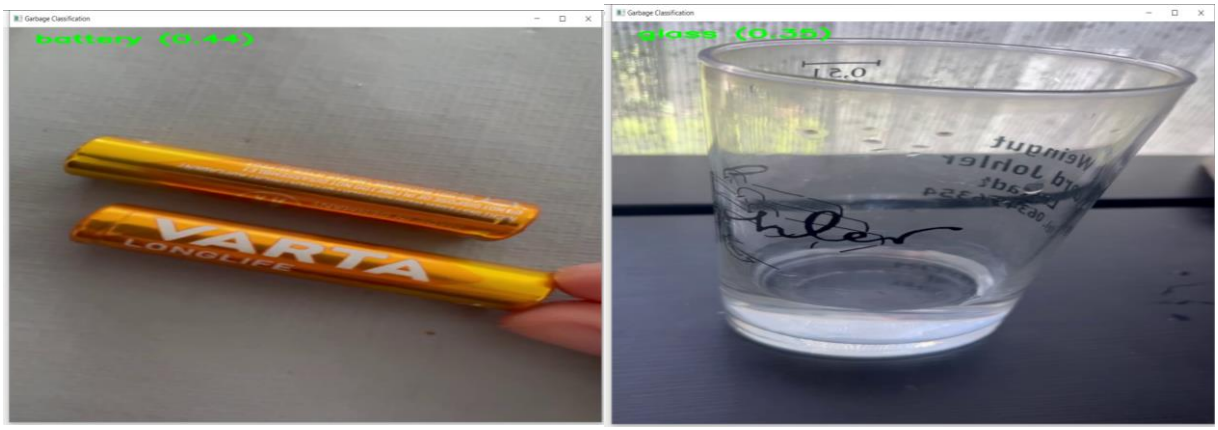


Рисунок 5.10 – Демонстрація розпізнавання сміття з відеофайлу

На основі результатів проведеного тестування можна видно, що розроблена мережа успішно впоралася із завданням сортування сміття. Вона продемонструвала стабільну та точну роботу при обробці зображень і відео, і під час розпізнавання об’єктів у режимі реального часу. Це свідчить про її ефективність, адаптивність до

різних типів вхідних даних і потенціал до практичного застосування в умовах реального середовища.

Висновки до розділу 5

У результаті проведених експериментів із навчання моделі для класифікації зображень за категоріями відходів було досягнуто прийнятного рівня точності. Після 53 епох навчання модель досягла тренувальної точності на рівні близько 85%, а валідаційна точність становила приблизно 78%. Це свідчить про ефективне навчання та задовільну здатність моделі до узагальнення.

Графік втрат показав стабільне зниження як тренувальних, так і валідаційних втрат. Відсутність різкого зростання валідаційних втрат наприкінці тренування вказує на те, що перенавчання не спостерігалося, а модель не втратила здатність до генералізації.

Аналіз матриці невідповідностей продемонстрував, що модель добре справляється з класифікацією таких класів, як clothes, cardboard, biological. Водночас виникали труднощі з класифікацією класів battery, metal та shoes.

Але загалом створена система успішно реалізує функцію класифікації різних типів відходів. Вона ефективно працює з різними джерелами даних — як зі статичних зображень, так і з відеопотоку у режимі реального часу, включно з веб-камерою.

Проведене тестування підтвердило стабільність і точність роботи моделі у різних умовах, що демонструє практичну придатність системи для автоматизованого сортування сміття.

Результати, наведені на рисунках 5.8 – 5.10 свідчать про високий рівень розпізнавання і надійність системи, що є важливим кроком для подальшого впровадження у реальних екологічних та промислових сценаріях.

Розроблена система готова до використання як ефективний інструмент для автоматизації процесів сортування відходів, що сприяє підвищенню екологічної свідомості та оптимізації управління ресурсами.

ВИСНОВКИ

Метою дипломного проєкту було підвищити якість роботизованої системи сортування сміття. Для цього було проаналізовано сучасні методи розпізнавання об’єктів, досліджено різні архітектури глибинних нейронних мереж, зокрема згорткові нейронні мережі, та обрано їх як основу для класифікатора. Для обробки зображень у режимі реального часу було застосовано бібліотеку OpenCV. Розробка системи здійснювалась з використанням мови програмування Python і фреймворків TensorFlow та Keras. Також було розглянуто існуючі аналоги систем сортування сміття та особливості їх реалізації.

Розроблено класифікатор, який розпізнає такі категорії сміття: батарейки, біологічні відходи, коробки, одяг, скло, метал, папір, пластик та взуття. Побудовано систему, яка реалізує цей класифікатор на основі згорткової нейронної мережі. Створено графічний інтерфейс на базі Tkinter, що дозволяє користувачам зручно взаємодіяти із системою, завантажувати зображення, відео та працювати з камерою. Проведено аналіз результатів роботи системи, зокрема побудовано графіки точності і втрат, матриці невідповідностей, що підтвердили якість роботи класифікатора з точністю понад 75%.

Розроблена система підвищує ефективність сортування сміття, дозволяючи автоматизувати процеси управління відходами. Вона має високу гнучкість та практичну цінність, легко адаптується для роботи на мобільних роботах та в автономних системах сортування. Завдяки модульній архітектурі її можна переносити на різні апаратні платформи з обмеженими ресурсами, наприклад, Raspberry Pi або NVIDIA Jetson з мінімальними змінами. Система може інтегруватись у промислові сортувальні станції та стати частиною екологічних ініціатив, сприяючи більш ефективному управлінню відходами в майбутньому.

Окрім того, архітектура розробленої системи передбачає подальше розширення функціоналу — зокрема додавання нових класів відходів, впровадження аналітичних модулів для збору статистики, а також інтеграцію з базами даних чи хмарними сервісами для централізованого моніторингу.

Запропонований підхід до автоматизації сортування має потенціал для масштабування в міських системах управління відходами, включаючи використання в освітніх або громадських ініціативах з підвищення екологічної свідомості. Отже, реалізований проєкт не лише відповідає поставленій меті, а й створює основу для майбутнього розвитку у сфері екологічних технологій і сталого розвитку.

ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Що таке комп’ютерний зір? URL: https://thetransmitted.com/adlucem/shho-take-kompyuternyj-zir-computer-vision-cv/ .
2. Support Vector Machines URL: https://scikit-learn.org/stable/modules/svm.html.
3. Machine Learning - K-nearest neighbors (KNN) URL: https://www.w3schools.com/python/python_ml_knn.asp.
4. Random Forest Algorithm in Machine Learning URL: https://www.geeksforgeeks.org/random-forest-algorithm-in-machine-learning/.
5. Кейс-стаді: семантична сегментація в прикладах і деталях URL: https://www.facerua.com/kieis-stadi-siemantichna-sieghmientatsiia-v-prikladakh-i-dietaliakh/.
6. What is a recurrent neural network? URL: https://www.ibm.com/think/topics/recurrent-neural-networks.
7. Згорткова нейронна мережа – просте пояснення CNN та її застосування URL: https://evergreens.com.ua/ua/articles/cnn.html.
8. Recurrent Neural Networks Explained URL: https://rahul-tawar.medium.com/recurrent-neural-networks-explained-b78db8c92810.
9. 3D візуалізація нейронної мережі URL: https://adamharley.com/nn_vis/cnn/3d.html.
10. Датасет Garbage Classification V2 URL: https://www.kaggle.com/datasets/sumn2u/garbage-classification-v2.
11. Garbage Classification URL: https://www.kaggle.com/datasets/mostafaabla/garbage-classification.
12. Classification_Garbage URL: https://www.kaggle.com/datasets/hipinhththu/classification-garbage.
13. Food Images (Food-101) URL: https://www.kaggle.com/datasets/kmader/food41
14. Garbage_classification_splitted URL: https://www.kaggle.com/datasets/minhluquang2/garbage-classification-splitted.

15. What is OpenCV Library? URL: https://www.geeksforgeeks.org/opencv-overview/.
16. What Is Tensorflow URL: https://www.nvidia.com/en-us/glossary/tensorflow/.
17. What is Keras? URL: https://www.geeksforgeeks.org/what-is-keras/.
18. Що таке нейронні мережі та де їх використовують? URL: https://incrypted.com/ua/shcho-take-nejromerezhi/.
19. Tutorial on Keras URL: flow_from_dataframehttps://vijayabhaskar96.medium.com/tutorial-on-keras-flow-from-dataframe-1fd4493d237c.
20. Liu W. A survey of deep neural network architectures and their applications / W. Liu, Z. Wang, X. Liu// Neurocomputing. — 2017.
21. С. О. Субботін. Нейронні мережі: теорія та практика: навч. посіб. Житомир: О. О. Євенок, 2020.
22. Що таке мобільний робот і які методи класифікації? URL: https://ua.roboct-tech.com/news/what-is-a-mobile-robot-and-what-are-the-classi-58489895.html.